

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий _____

Кафедра Информационные системы и технологии _____

Специальность 6-05-0612-01 Программная инженерия (профилизация
Программное обеспечение информационных технологий) _____

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ НА ТЕМУ:**

«Реализация базы данных системы бронирования билетов для маршрутных
транспортных средств с применением технологии шифрования и маскирования в
БД»

Выполнил студент _____ Хуторцов Кирилл Владимирович
(Ф.И.О.)

Руководитель работы _____ асс. Подрез А.А.
(учен. степень, звание, должность, Ф.И.О., подпись)

Зав. кафедрой _____ к. т. н. Блинова Е.А.
(учен. степень, звание, должность, Ф.И.О., подпись)

Курсовая работа защищена с оценкой _____

Минск 2025

Содержание

Введение	4
1 Постановка задачи	5
1.1 Общая постановка задачи	5
1.2 Обзор аналогичных решений	6
1.3 Обзор средств разработки	11
1.4 Описание применяемой технологии	11
1.5 Выводы по разделу	11
2 Проектирование базы данных	13
2.1 Организация базы данных	13
2.2 Выводы по разделу	16
3 Разработка объектов базы данных	18
3.1 Разработка индексов базы данных	18
3.2 Разработка процедур базы данных	18
3.3 Разработка механизмов целостности (триггерная логика)	19
3.4 Разработка пользователей	19
3.5 Вывод по разделу	20
4 Описание процедур импорта и экспорта	21
4.1 Реализация экспорта данных в формат JSON	21
4.2 Реализация импорта данных из формата JSON	21
4.3 Вывод по разделу	23
5 Тестирование производительности	24
5.1 Подготовка тестового стенда и анализ нагрузки	24
5.2 Выявление проблемы: полное сканирование	24
5.3 Решение: применение покрывающего индекса	25
6 Описание технологии и её применение в базе данных	27
6.1 Динамическое маскирование данных	27
6.2 Шифрование в БД	28
6.3 Защита аутентификационных данных пользователей	29

6.4 Вывод по разделу	29
7 Краткое описание приложения для демонстрации	30
8 Руководство пользователя	33
8.1 Сценарий обычного пользователя	33
8.2 Сценарий администратора	38
8.3 Вывод по разделу	41
Заключение	42
Приложение А	43
Приложение Б	44
Приложение В	46
Приложение Г	80
Приложение Д	84

Введение

В современном мире цифровых технологий рынок пассажирских перевозок претерпевает значительные изменения, где онлайн-платформы для бронирования билетов играют ключевую роль в соединении пассажиров и транспортных компаний. Такие системы позволяют эффективно управлять маршрутами, расписаниями, водителями, бронированиями и личными данными пользователей, обеспечивая удобство, скорость и точность планирования поездок. Однако для успешного функционирования подобных платформ необходима надёжная и оптимизированная база данных, способная обрабатывать большие объёмы транзакций, обеспечивать безопасность доступа и поддерживать аналитические функции для извлечения ценной информации о спросе и загрузке маршрутов.

Курсовая работа посвящена реализации базы данных для программного средства «Бронирование билетов на маршрутные транспортные средства» с применением технологии шифрования и маскирования в БД. Тема актуальна в контексте растущего спроса на безопасные и защищённые системы управления данными, где шифрование и маскирование позволяют предотвращать утечки конфиденциальной информации. В рамках работы будет разработана база данных на основе СУБД MS SQL Server, обеспечивающая управление ролями пользователей, маршрутами, водителями, рейсами и бронированиями. Особое внимание уделяется интеграции технологий импорта/экспорта данных в формате JSON, тестированию производительности на больших объёмах данных и анализу планов запросов для повышения эффективности системы.

Целью курсовой работы является проектирование и реализация базы данных, интегрирующей механизмы шифрования и динамического маскирования данных, для демонстрации практического применения концепций безопасности и производительности в реальной информационной системе. В процессе выполнения будут решены задачи по созданию объектов базы данных, обеспечению доступа исключительно через хранимые процедуры, тестированию и оптимизации, а также описанию применения выбранных технологий защиты. Результаты работы позволят продемонстрировать, как базы данных с продвинутыми механизмами безопасности способствуют развитию программного обеспечения в области транспортных информационных технологий.

1 Постановка задачи

1.1 Общая постановка задачи

Целью курсовой работы является проектирование, реализация и тестирование базы данных для программного средства «Бронирование билетов на маршрутные транспортные средства» с применением технологии шифрования и маскирования в БД на платформе СУБД Microsoft SQL Server. Система должна обеспечивать безопасное и эффективное управление информацией о маршрутах, водителях, рейсах, бронированиях и пользователях, а также гарантировать защиту конфиденциальных данных и высокую производительность при массовых операциях.

Основные задачи:

1. Разработка логической и физической модели базы данных, включающей:
 - определение ролей пользователей: «администратор», «пользователь»;
 - управление маршрутами (добавление, изменение, удаление);
 - управление водителями (добавление, изменение, удаление);
 - обеспечение бронирования билетов;
 - анализ маршрутов (количество забронированных билетов, общее количество маршрутов).
2. Реализация объектов базы данных в MS SQL Server:
 - создание таблиц, представлений, индексов, ограничений целостности;
 - разработка хранимых процедур как единственного механизма доступа к данным (принцип инкапсуляции);
 - обеспечение безопасности доступа через роли, шифрование и маскирование данных.
3. Обеспечение обмена данными в формате JSON:
 - импорт данных о бронированиях из JSON-файлов;;
 - экспорт данных о бронированиях в JSON для интеграции с внешними системами или веб-приложением.
4. Тестирование производительности:
 - заполнение ключевой таблицы данными объёмом не менее 100 000 строк;
 - анализ планов выполнения запросов для типичных операций;
 - оптимизация структуры.
5. Применение технологии шифрования и маскирования:
 - интеграция механизмов безопасности с использованием встроенных средств SQL Server;
 - реализация защиты чувствительных данных:
 - шифрование номеров водительских удостоверений;
 - маскирование ФИО водителей;
 - хеширование паролей;
 - описание используемых системных компонентов (ключей шифрования, сертификатов, политик доступа) и демонстрация их применения в БД для соответствия стандартам защиты персональных данных.

1.2 Обзор аналогичных решений

Немаловажным этапом разработки информационной системы является сравнительный анализ существующих решений. В рамках данного курсового проекта основной упор делается на проектирование надежной архитектуры базы данных, поэтому анализ прототипов проводился с позиции организации хранения данных, безопасности и реализации бизнес-логики на уровне СУБД.

В качестве первого аналога был рассмотрен веб-сервис «INFOBUS.BY», специализирующийся на бронировании автобусных билетов. Данная платформа предоставляет пользователям возможность поиска и покупки билетов на междугородние и международные автобусные маршруты, интегрируясь с расписаниями перевозчиков. С точки зрения структуры БД, система оперирует сложными связями между перевозчиками, маршрутами и динамическими схемами мест в транспортных средствах.

Плюсы:

- реализация сложной логики визуализации мест (требует хранения координат или индексов мест для каждой модели автобуса);
- интеграция с внешними API для синхронизации расписаний в реальном времени.

Минусы:

- сложная процедура отмены транзакций, что указывает на возможные избыточные блокировки в БД при обработке возвратов.

Чтобы найти билеты, нужно заполнить поля с направлениями и датой. Далее нужно дождаться конца поиска, чтобы увидеть все доступные варианты, в том числе самые быстрые и дешёвые.

Интерфейс поиска демонстрирует необходимость реализации в БД эффективных процедур фильтрации по датам и городам и представлен на рисунке 1.1.

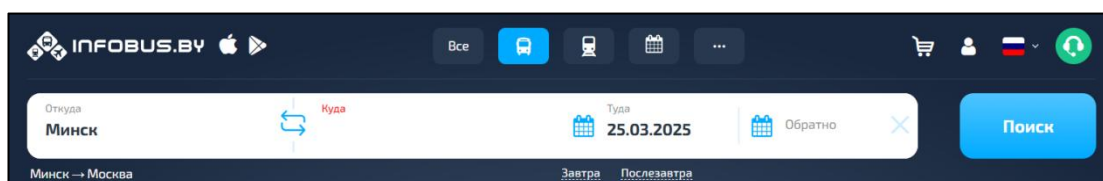


Рисунок 1.1 — Поиск билета в приложении «INFOBUS.BY»

Отображение результатов требует от СУБД выполнения тяжелых JOIN-запросов между таблицами рейсов, маршрутов и перевозчиков в реальном времени. Интерфейс для отображения результатов поиска представлен на рисунке 1.2.

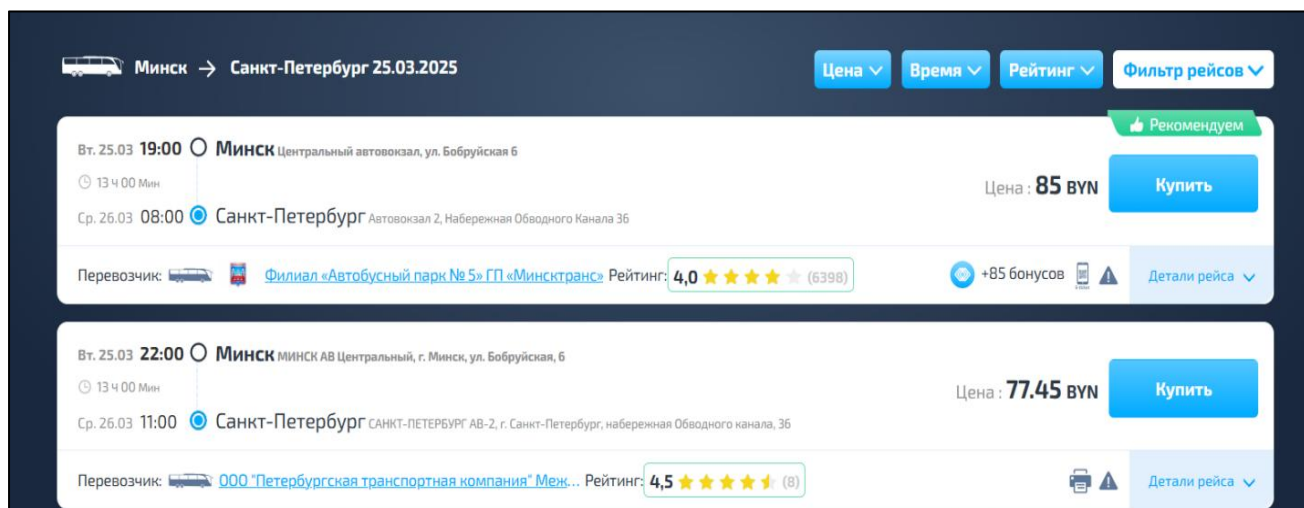


Рисунок 1.2 — Результат поиска билета в приложении «INFOBUS.BY»

В качестве второго аналога рассмотрен веб-сервис «SMILEBUS.BY», специализирующийся на бронировании автобусных билетов. Данная платформа предоставляет пользователям возможность поиска и покупки билетов между городами и городскими поселками, интегрируясь с расписаниями перевозчиков. Прототип интересен с точки зрения реализации программ лояльности и управления очередями на уровне базы данных.

Плюсы:

- наличие подсистемы «очереди» (реализуется через триггеры или очереди сообщений в БД при освобождении места);
- сложная логика расчета стоимости (скидки, бонусы), требующая выноса вычислений в хранимые функции.

Минусы:

- жесткие ограничения (санкции) за отмену брони, что требует ведения в БД детальной истории «репутации» каждого UserID.

Форма поиска и результаты подтверждают важность индексации полей отправления и прибытия для обеспечения высокой скорости отклика системы. Форма для поиска билетов представлена на рисунке 1.3.

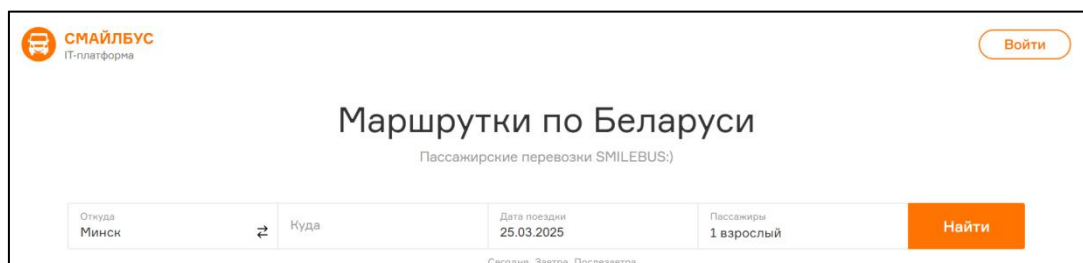


Рисунок 1.3 — Поиск билета в приложении «SMILEBUS.BY»

Результаты поиска маршрута по введенным пользователем данным представлены на рисунке 1.4.

Откуда: Минск → Куда: Глуск Дата поездки: 08.04.2025 Пассажиры: 1 взрослый Найти

Сегодня Завтра Послезавтра

Место посадки: Минск, м. Институт Культуры Место высадки: Глуск, ПМК-249

Выбрать на карте

Отправление	Прибытие	Цена	Места
Минск - Глуск - Октябрьский 11:00 Минск, м. Институт Культуры	13:32 Глуск, ПМК-249	2 часа 32 мин в пути 28 р.	Мест нет Стать в очередь
Минск - Глуск - Октябрьский 14:00 Минск, м. Институт Культуры	16:32 Глуск, ПМК-249	2 часа 32 мин в пути 28 р.	Мест нет Стать в очередь
Минск - Глуск - Октябрьский 18:00 Минск, м. Институт Культуры	20:32 Глуск, ПМК-249	2 часа 32 мин в пути 28 р.	3+ Заказать

Рисунок 1.4 — Результат поиска билета в приложении «SMILEBUS.BY»

Рассматриваемое программное средство также предлагает возможность сохранения истории заказов пользователя. Пример представлен на рисунке 1.5.

История заказов Бонусные баллы 2 210 Настройки Статус «Хороший»

Все заказы Будущие Прошедшие

14 апреля, понедельник 08:24 Остановки на карте	Глуск Пов. Достижение Минск ул. Аранская	3 взрослых 84 р.	Заказ не подтвержден Подтверждение будет доступно через 18 дней 14 часов 22 минуты 7 секунд	Редактировать Отменить заказ
12 апреля, суббота 14:07 Остановки на карте	Минск ул. Аранская Глуск Пов. Достижение	3 взрослых 84 р.	Заказ не подтвержден Подтверждение будет доступно через 16 дней 20 часов 32 минуты 7 секунд	Редактировать Отменить заказ

Рисунок 1.5 — Хранение истории заказов в приложении «SMILEBUS.BY»

Система авторизации в данном аналоге подчеркивает необходимость безопасного хранения персональных данных, что в моем проекте будет решено через хеширование паролей.

Рисунок 1.6 — Система регистрации и авторизации в приложении «SMILEBUS.BY»

В качестве третьего аналога рассмотрен веб-сервис «ATLASBUS.BY», специализирующийся на бронировании автобусных билетов. Данная платформа предоставляет пользователям возможность поиска и покупки билетов на междугородние и международные автобусные маршруты, интегрируясь с расписаниями перевозчиков. Система выделяется глубокой проработкой управления статусами бронирования.

Плюсы:

- развитая система статусов брони (подтверждено, ожидание, отменено), что требует реализации транзакционной целостности;
- гибкое управление справочниками маршрутов.

Минусы:

- проблемы с актуальностью данных («фантомные» рейсы), что говорит о недостатках в процедурах проверки доступности мест перед выдачей ответа.

На главной странице сайта пользователь может осуществить поиск интересующего его маршрута. Форма поиска и фильтрация результатов наглядно показывают потребность в использовании динамических SQL-запросов или параметризованных хранимых процедур. Форма для поиска билета представлена на рисунке 1.7.

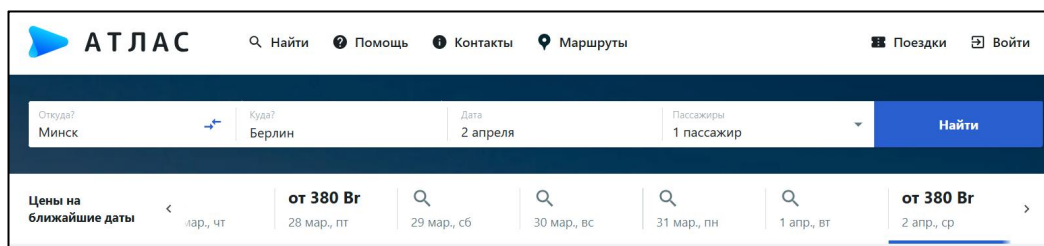


Рисунок 1.7 — Поиск билета на сайте «ATLASBUS.BY»

Результаты поиска маршрута по введенным пользователем данным представлены на рисунке 1.8.

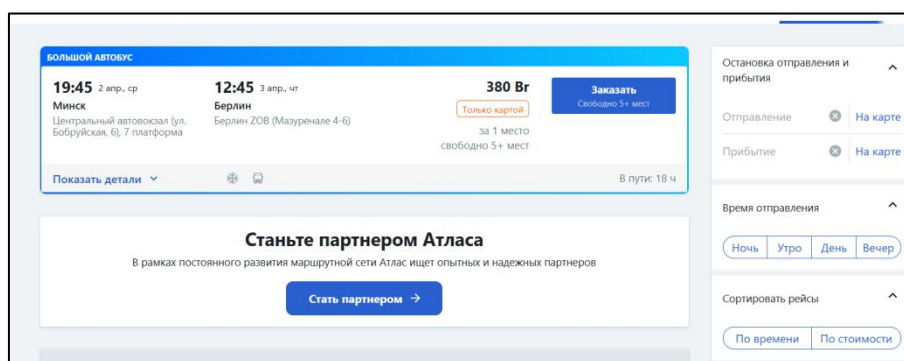


Рисунок 1.8 — Результат поиска билета на сайте «ATLASBUS.BY»

Как видно на рисунке, результаты поиска можно сортировать по различным критериям.

Рассматриваемый сайт также поддерживает систему регистрации и авторизации пользователей. Форма для регистрации/авторизации представлена на рисунке 1.9.

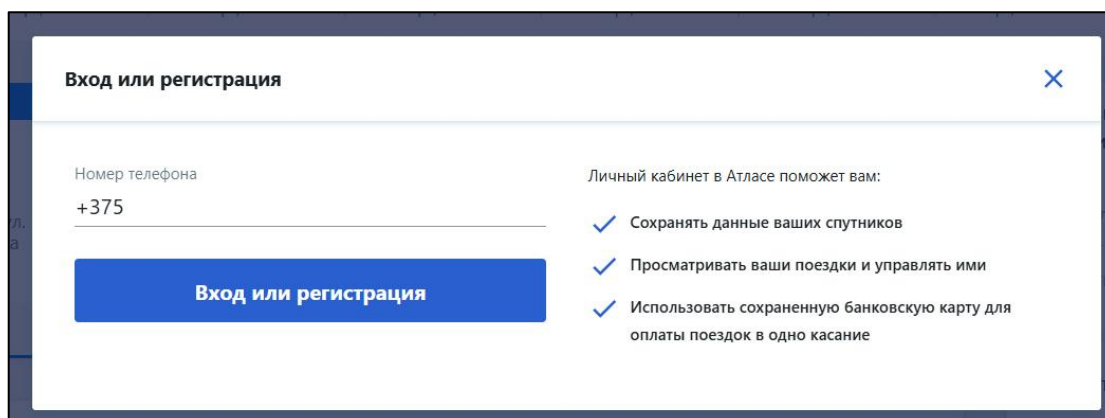


Рисунок 1.9 — Система регистрации и авторизации на сайте «ATLASBUS.BY»

Анализ прототипов показал, что критически важными аспектами являются не только удобство интерфейса, но и надежность хранения данных. Большинство минусов аналогов (проблемы с отменами, актуальностью данных) связаны с ошибками в логике на стороне сервера БД.

1.3 Обзор средств разработки

Современные средства разработки предоставляют разработчику единое рабочее пространство, где можно писать код, выполнять его, анализировать ошибки, подключаться к внешним сервисам и базам данных, а также управлять проектом. Благодаря этому снижается количество рутинных операций, повышается качество программного обеспечения и ускоряется процесс разработки.

Microsoft SQL Server — это мощная система управления реляционными базами данных (СУБД), широко используемая в корпоративных приложениях. Она предоставляет полный набор инструментов для проектирования, создания, администрирования и оптимизации баз данных, ключевыми особенностями которой являются возможность создания таблиц, индексов, представлений и хранимых процедур, анализ производительности и управления пользователями и ролями.

Visual Studio Code (VS Code) — это современный, лёгкий и расширяемый редактор кода, который стал одним из самых популярных инструментов среди разработчиков. Он не является полноценной IDE, но благодаря системе расширений может выполнять функции, сравнимые с большими средами разработки.

В контексте проекта VS Code используется для разработки клиентского приложения на Node.js + Express и создания визуальной части приложения на React.

1.4 Описание применяемой технологии

В рамках разработки проекта была реализована технология шифрования и маскирования в БД. Особое внимание было уделено использованию бинарных форматов данных, которые позволяют хранить зашифрованную информацию непосредственно в базе данных. В проектируемой системе это реализовано через шифрование на уровне ячеек (Cell-Level Encryption), где критически важные сведения, такие как номера лицензий водителей, защищаются алгоритмом AES_256 с использованием сертификатов и симметричных ключей. Параллельно с этим внедрена технология динамического маскирования данных (Dynamic Data Masking) для обеспечения приватности персональной информации, например, имен водителей, что позволяет скрывать реальные значения от пользователей без соответствующих прав доступа. Такой комплексный подход на базе СУБД MS SQL Server гарантирует целостность и безопасность данных внутри системы бронирования, исключая необходимость использования внешних криптографических модулей и упрощая общую архитектуру приложения.

1.5 Выводы по разделу

В рамках данного раздела было проведено комплексное исследование предметной области, позволившее сформировать прочный фундамент для

проектирования системы бронирования билетов. Глубокий сравнительный анализ ведущих рыночных решений, таких как «INFOBUS.BY», «SMILEBUS.BY» и «ATLASBUS.BY», выявил критическую зависимость качества сервиса от архитектуры базы данных. Установлено, что типичные проблемы конкурентов — задержки при обновлении статусов рейсов и уязвимость персональных данных — эффективно решаются за счет переноса бизнес-логики на сторону СУБД MS SQL Server.

Особое внимание в проекте уделено парадигме безопасной разработки. Обосновано применение комбинированного подхода к защите информации: использование технологии Dynamic Data Masking для сокрытия персональных данных в публичных интерфейсах и Cell-Level Encryption для криптографической защиты документов. Выбор хранимых процедур как единственного интерфейса взаимодействия с данными реализует принцип инкапсуляции, что минимизирует риски SQL-инъекций и обеспечивает целостность транзакций при массовом бронировании.

Технологический стек, включающий MS SQL Server для хранения данных и Node.js для реализации бизнес-логики приложения, позволяет достичь необходимого баланса между безопасностью и производительностью. Использование формата JSON для импорта и экспорта данных гарантирует высокую масштабируемость и легкость интеграции с внешними информационными системами. Сформулированные требования и выбранные методы реализации создают предпосылки для разработки высокопроизводительной системы, способной эффективно обрабатывать массивы данных объемом свыше 100 000 записей, сохраняя при этом конфиденциальность информации и высокую скорость отклика для конечного пользователя.

2 Проектирование базы данных

2.1 Организация базы данных

База данных для программного средства «Бронирование билетов на маршрутные транспортные средства» реализована в СУБД Microsoft SQL Server и предназначена для хранения и обработки информации о пользователях (пассажирах и администраторах), маршрутах, водителях, рейсах и бронированиях. Структура обеспечивает нормализацию до 3НФ, целостность данных, безопасность доступа через роли и хранимые процедуры, шифрование конфиденциальных данных (Always Encrypted), маскирование персональных данных (Dynamic Data Masking), а также импорт/экспорт данных в формате JSON.

Всего в базе данных создано 5 основных таблиц, связанных внешними ключами. Доступ к данным осуществляется исключительно через хранимые процедуры, что соответствует требованиям задания.

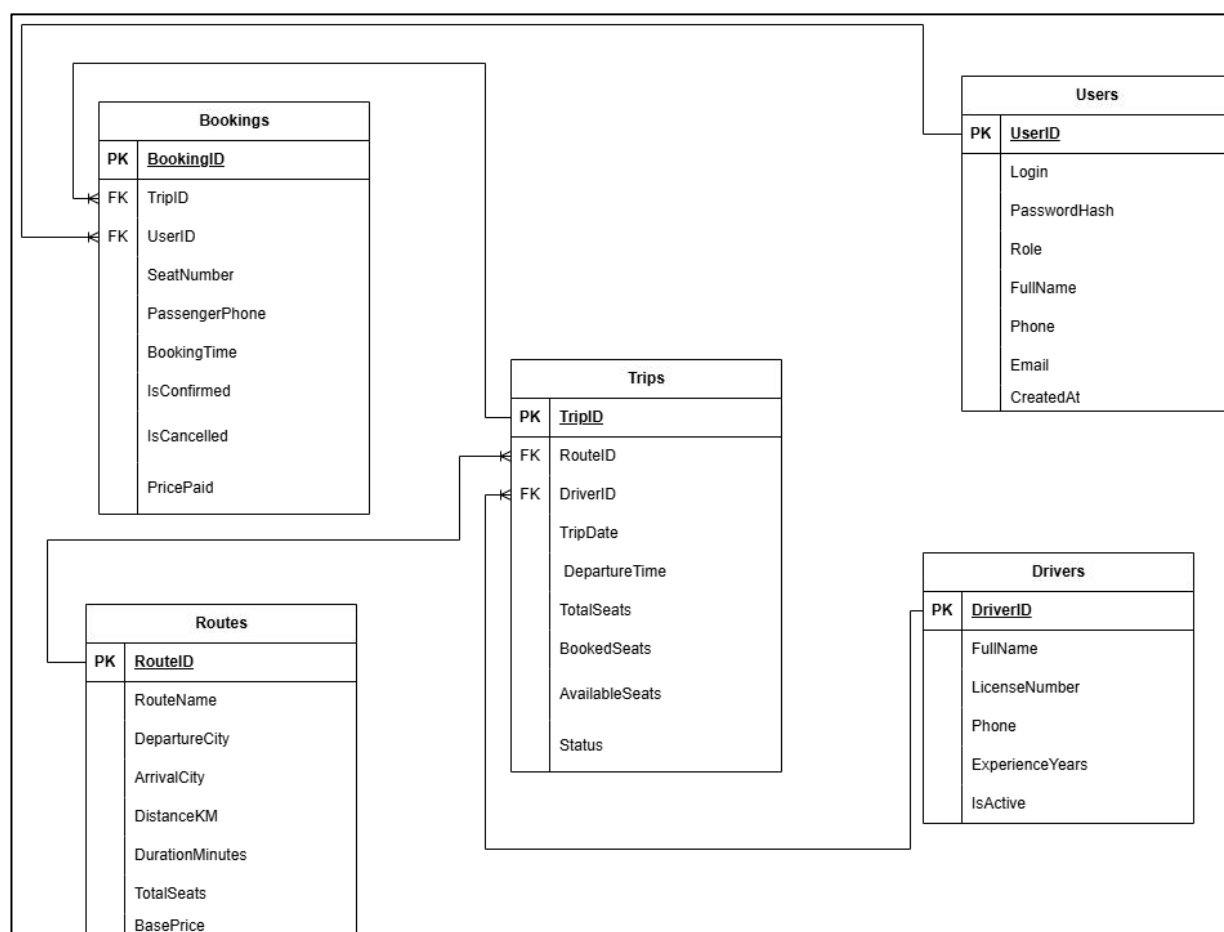


Рисунок 2.1 – Диаграмма структуры спроектированной базы данных

Таблица Users предназначена для хранения общей информации о всех пользователях системы — единой точки входа для аутентификации и определения ролей. Является родительской таблицей для бронирований. Структура таблицы представлена в таблице 2.1.

Таблица 2.1 – Структура таблицы Users

Столбец	Тип данных	Ограничение целостности
UserId	INT IDENTITY(1,1)	PRIMARY KEY
Login	NVARCHAR(50)	NOT NULL, UNIQUE
PasswordHash	VARBINARY(64)	NOT NULL
Role	NVARCHAR(20)	CHECK (Role IN ('admin', 'user')), NOT NULL
FullName	NVARCHAR(100)	
Phone	NVARCHAR(15)	
Email	NVARCHAR(100)	
CreatedAt	DATETIME2	DEFAULT SYSDATETIME()

Таблица Routes предназначена для хранения информации о маршрутах — города отправления и прибытия, расстояние, длительность, количество мест, базовая цена. Является родительской для рейсов. Структура таблицы представлена в таблице 2.2.

Таблица 2.2 – Структура таблицы Routes

Столбец	Тип данных	Ограничение целостности
RouteID	INT IDENTITY(1,1)	PRIMARY KEY
RouteName	NVARCHAR(100)	NOT NULL
DepartureCity	NVARCHAR(50)	NOT NULL
ArrivalCity	NVARCHAR(50)	NOT NULL
DistanceKM	INT	CHECK (DistanceKM > 0)
DurationMinutes	INT	CHECK (DurationMinutes > 0)
TotalSeats	INT	CHECK (TotalSeats BETWEEN 10 AND 60), NOT NULL
BasePrice	DECIMAL(10,2)	CHECK (BasePrice > 0), NOT NULL

Таблица Drivers предназначена для хранения данных о водителях — номер лицензии (зашифрован), ФИО (маскирован), стаж, статус. Столбец LicenseNumber защищён с помощью Always Encrypted (Deterministic). Структура таблицы представлена в таблице 2.3.

Таблица 2.3 – Структура таблицы Drivers

Столбец	Тип данных	Ограничение целостности
DriverID	INT IDENTITY(1,1)	PRIMARY KEY
FullName	NVARCHAR(100)	MASKED WITH (FUNCTION = 'partial(1, "XXXX", 0)') NOT NULL
LicenseNumber	VARBINARY(MAX)	NOT NULL
Phone	NVARCHAR(15)	NOT NULL
ExperienceYears	INT	CHECK (ExperienceYears >= 0)
IsActive	BIT	DEFAULT 1

Таблица Trips хранит данные о конкретных рейсах — маршрут, водитель, дата, время, количество мест, статус. Является связующей между Routes, Drivers и Bookings. Структура таблицы представлена в таблице 2.4.

Таблица 2.4 – Структура таблицы Trips

Столбец	Тип данных	Ограничение целостности
TripID	INT IDENTITY(1,1)	PRIMARY KEY
RouteID	INT	NOT NULL
DriverID	INT	NOT NULL
TripDate	DATE	NOT NULL
DepartureTime	TIME	NOT NULL
TotalSeats	INT	NOT NULL
BookedSeats	INT	DEFAULT 0
AvailableSeats	AS (TotalSeats - BookedSeats)	PERSISTED
Status	NVARCHAR(20)	DEFAULT 'planned', CHECK (Status IN ('planned', 'in_progress', 'completed', 'cancelled'))
		CONSTRAINT CHK_TripSeats CHECK (TotalSeats BETWEEN 10 AND 60)
		CONSTRAINT FK_Trips_Routes FOREIGN KEY (RouteID) REFERENCES Routes(RouteID)

Таблица Bookings предназначена для хранения бронирований — ключевой элемент взаимодействия. Подлежит тестированию на 100 000+ строк. Структура таблицы представлена в таблице 2.5.

Таблица 2.5 – Структура таблицы Bookings

Столбец	Тип данных	Ограничение целостности
BookingID	BIGINT IDENTITY(1,1)	PRIMARY KEY
TripID	INT	NOT NULL
UserID	INT	NOT NULL
SeatNumber	INT	NOT NULL
PassengerPhone	NVARCHAR(15)	
IsCancelled	BIT	DEFAULT 0
BookingTime	DATETIME2	DEFAULT SYSDATETIME()
IsConfirmed	BIT	DEFAULT 1
PricePaid	DECIMAL(10,2)	NOT NULL
		CONSTRAINT FK_Bookings_Trips FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
		CONSTRAINT FK_Bookings_Users FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE NO ACTION

Представленная структура базы данных обеспечивает надёжное, безопасное и масштабируемое хранение данных, поддерживает роли пользователей, управление маршрутами, водителями, рейсами и бронированиями, а также анализ загрузки маршрутов.

В рамках этой базы данных предусмотрено создание двух ролей: администратора и пользователя, каждая из которых имеет свой набор прав и возможностей. Диаграмма вариантов использования для ролей администратора и пользователя представлена в приложении А, код создания таблиц — в приложении Б. Ключевым элементом для управления доступом к данным и выполнением операций в базе данных являются хранимые процедуры.

2.2 Выводы по разделу

В представленном разделе было проведено комплексное проектирование реляционной структуры базы данных, которая выступает фундаментом для функционирования информационной системы бронирования билетов. В основу разработки легла пятизвенная модель данных, включающая таблицы пользователей, маршрутов, водителей, рейсов и бронирований, что позволило достичь нормализации данных до уровня третьей нормальной формы и исключить

нежелательную избыточность. Спроектированная архитектура не только обеспечивает логическую целостность связей через систему первичных и внешних ключей, но и закладывает технологический базис для обеспечения высокой производительности при масштабировании системы до уровня обработки сотен тысяч транзакций.

Особое значение в структуре отведено реализации механизмов комплексной безопасности на уровне ядра СУБД. Интеграция технологий шифрования Always Encrypted для защиты документов и динамического маскирования данных для персональной информации позволяет системе соответствовать современным стандартам защиты персональных данных. Применение вычисляемых столбцов, таких как состояние доступности мест на рейсе, переносит часть вычислительной нагрузки на сервер базы данных, обеспечивая мгновенный отклик при поиске билетов.

Важнейшим архитектурным решением стало разделение полномочий через систему ролей администратора и пользователя, а также полная инкапсуляция логики доступа в хранимых процедурах. Такой подход гарантирует, что взаимодействие с данными происходит исключительно по строго заданным алгоритмам, предотвращая несанкционированные изменения и обеспечивая единство бизнес-правил для всех программных модулей. Таким образом, сформированная логическая и физическая модель базы данных является надежной и защищенной средой, готовой к интеграции с прикладным интерфейсом и массовому тестированию производительности.

3 Разработка объектов базы данных

3.1 Разработка индексов базы данных

Индекс в базе данных представляет собой объект, который используется для ускорения поиска данных. Если таблица содержит большое количество строк, то последовательный поиск данных может занимать много времени.

В базе данных автоматически созданы кластеризованные индексы для всех первичных ключей. Однако, для оптимизации работы с таблицей Bookings, которая подвергается наибольшей нагрузке при поиске, были разработаны дополнительные некластеризованные индексы.

Например, для быстрого поиска бронирований по рейсу и времени создан покрывающий индекс IX_Bookings_Trip_Time. Создание индекса представлено в листинге 3.2.

```
CREATE NONCLUSTERED INDEX IX_Bookings_Trip_Time ON Bookings (TripID,  
BookingTime)  
INCLUDE (PricePaid, IsCancelled);
```

Листинг 3.2 –Индекс для таблицы Bookings

Применение индексов в базе данных значительно повышает скорость выполнения операций поиска, сортировки и фильтрации данных, особенно в случае больших объемов информации (более 100 000 строк в таблице бронирований).

3.2 Разработка процедур базы данных

Хранимые процедуры являются основным интерфейсом взаимодействия приложения с базой данных. Они инкапсулируют логику манипулирования данными и обеспечивают безопасность, предотвращая прямой доступ к таблицам.

Всего было разработано около 30 процедур, которые можно разделить на группы: CRUD-операции для пользователей, водителей, маршрутов, рейсов и бронирований; аналитические процедуры; процедуры импорта/экспорта в JSON. На листинге 3.3 представлен пример части реализации одной из процедур, листинг остальных процедур представлен в приложении В.

```
BEGIN TRANSACTION  
BEGIN TRY  
    INSERT INTO Bookings (TripID, UserID, SeatNumber, PassengerPhone,  
PricePaid, IsCancelled)  
    VALUES (@TripID, @UserID, @SeatNumber, @PassengerPhone, @Price,  
0);  
  
    UPDATE Trips
```

```

SET BookedSeats = BookedSeats + 1
WHERE TripID = @TripID;

COMMIT TRANSACTION;
SELECT 1 AS Success, N'Бронирование успешно.' AS Message;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SELECT 0 AS Success, ERROR_MESSAGE() AS Message;
END CATCH

```

Листинг 3.3 – Часть процедуры бронирования билета

Данная процедура не просто вставляет данные, но и выполняет проверку бизнес-логики (доступность мест, отсутствие дубликатов), а также использует транзакции для обеспечения целостности. Скрипт других процедур приведен в приложении к курсовой работе.

3.3 Разработка механизмов целостности (триггерная логика)

Вместо использования классических DML-триггеров, в данной базе данных применена декларативная ссылочная целостность с каскадным удалением. Это более производительный подход для высоконагруженных систем.

В частности, при создании таблиц использована конструкция ON DELETE CASCADE. Это гарантирует, что при удалении маршрута из таблицы Routes автоматически удаляются связанные рейсы в Trips и бронирования в Bookings, что исключает появление «сиротских» записей.

Пример реализации каскадного удаления в определении внешнего ключа приведен в листинге 3.4.

```

ALTER TABLE Trips
ADD CONSTRAINT FK_Trips_Routes FOREIGN KEY (RouteID) REFERENCES
Routes(RouteID) ON DELETE CASCADE;

```

Листинг 3.4 – Реализация каскадного удаления

Для более сложной логики, например, при удалении пользователя, используется транзакционная хранимая процедура sp_DeleteUser, которая выполняет очистку данных в строго определенном порядке, обеспечивая атомарность операции и проверку на наличие активных бронирований.

3.4 Разработка пользователей

Пользователь базы данных — это физическое или юридическое лицо, которое имеет доступ к БД. В проектируемой системе управление доступом

реализовано на уровне приложения, однако база данных поддерживает ролевую модель через структуру таблицы Users.

При проектировании было выделено 2 типа пользователей:

1. Пользователь: имеет право регистрироваться, аутентифицироваться, искать маршруты, бронировать билеты и просматривать бронирования.
2. Администратор: имеет полные права на управление контентом, включая создание/обновление/удаление водителей, маршрутов, а также доступ к аналитике и импорту/экспорту данных.

Разделение прав реализовано логически через поле Role в таблице Users. Это позволяет гибко настраивать права доступа, не создавая отдельные учетные записи СУБД для каждого пользователя приложения, что упрощает администрирование системы при большом количестве пользователей.

3.5 Вывод по разделу

В данном разделе была реализована система программных объектов базы данных, обеспечивающая эффективное управление всеми бизнес-процессами системы бронирования. Ключевым этапом физического проектирования стало создание системы некластеризованных индексов, которые позволили существенно сократить время поиска и фильтрации данных в высоконагруженных таблицах объемом более 100 000 записей. Применение таких структур критически важно для обеспечения мгновенного отклика интерфейса при массовых запросах пользователей в режиме реального времени.

Особое внимание было уделено инкапсуляции логики манипулирования данными внутри хранимых процедур. Это решение позволило не только защитить систему от внешних угроз, таких как SQL-инъекции, но и гарантировать строгое соблюдение бизнес-правил за счет использования транзакционных механизмов. Атомарность операций при бронировании билетов и автоматическое обновление счетчиков свободных мест исключают риск возникновения противоречивых данных даже при высокой конкурентной нагрузке.

Завершающим элементом архитектуры стала реализация механизмов ссылочной целостности и ролевой модели доступа. Сочетание каскадных правил удаления и логического разграничения прав администраторов и пользователей создало гибкую, масштабируемую и защищенную среду. В совокупности разработанные объекты формируют надежный фундамент базы данных, полностью соответствующий требованиям производительности и безопасности, заявленным в техническом задании.

4 Описание процедур импорта и экспорта

4.1 Реализация экспорта данных в формат JSON

Для обеспечения возможности обмена данными о бронированиях с внешними системами в системе разработана процедура экспорта .

Основной задачей процедуры является формирование полного, денормализованного набора данных о каждой транзакции, включая информацию из связанных таблиц. Такой подход позволяет получить в выгрузке не только идентификаторы, но и человекочитаемые данные: название маршрута, логин пользователя, стоимость.

Методологической основой экспорта является встроенная функциональность T-SQL FOR JSON PATH. Она преобразует реляционный результат запроса в иерархическую структуру JSON напрямую на стороне сервера, что является высокопроизводительным решением. Доступ к процедуре строго регламентирован и требует прав администратора, поскольку она выгружает конфиденциальную информацию о бронированиях.

Запрос к базе данных объединяет ключевые таблицы для формирования контекста бронирования:

```
SELECT
    B.BookingID,
    R.RouteName,
    U.Login AS UserLogin,
    B.SeatNumber,
    B.BookingTime,
    B.PricePaid
FROM Bookings B
JOIN Trips T ON B.TripID = T.TripID
JOIN Routes R ON T.RouteID = R.RouteID
JOIN Users U ON B.UserID = U.UserID
ORDER BY B.BookingTime DESC;
```

Листинг 4.1 – Фрагмент формирования денормализованного набора данных для экспорта

Полученный денормализованный набор данных возвращается вызывающему приложению, где может быть легко сериализован в единый JSON-документ, обеспечивая полноту информации о каждой транзакции для дальнейшего использования.

4.2 Реализация импорта данных из формата JSON

Для поддержки высокопроизводительных операций и массовой загрузки данных в систему разработана процедура импорта. Ключевой особенностью реализации является отказ от медленных курсоров в пользу Set-based operations и

использование временного буфера для обработки больших объемов данных .

Процедура принимает на вход строку формата JSON и выполняет ряд оптимизированных этапов обработки. Входящая JSON-строка сначала парсится и буферизуется с помощью функции OPENJSON с явным определением схемы. Полученные данные немедленно загружаются во временную таблицу. На этом же этапе происходит обогащение данных и первичная проверка их целостности. Затем в буферной таблице выполняется валидация и дедупликация, очищающая пакет от дубликатов, сгенерированных внутри загружаемого массива. Финальный этап — массовая вставка, для которой используется вызов оптимизированной процедуры sp_BulkInsertBookings. Эта процедура выполняет вставку всех строк сразу в рамках одной транзакции, используя хинт TABLOCK для минимизации логирования и достижения максимальной скорости.

Процесс парсинга и начальной загрузки в буфер выглядит следующим образом:

```
INSERT INTO #BulkBookings (TripID, UserID, SeatNumber,
PassengerPhone, PricePaid)
SELECT
    B.TripID,
    B.UserID,
    B.SeatNumber,
    B.PassengerPhone,
    ISNULL(R.BasePrice, 0)
FROM OPENJSON(@JsonData, '$.Bookings')
WITH (
    TripID INT '$.TripID',
    UserID INT '$.UserID',
    SeatNumber INT '$.SeatNumber',
    PassengerPhone NVARCHAR(20) '$.PassengerPhone'
) B
INNER JOIN Trips T ON T.TripID = B.TripID
INNER JOIN Routes R ON R.RouteID = T.RouteID;
```

Листинг 4.2 – Использование OPENJSON и временной таблицы для буферизации данных

Финальным этапом импорта является пересчет и обновление агрегированных данных о занятых местах в таблице Trips. Это выполняется одним обновляющим запросом для всех затронутых рейсов, что значительно снижает общую нагрузку на сервер, демонстрируя приверженность принципам оптимизации производительности.

4.3 Вывод по разделу

В ходе разработки объектов данного раздела были реализованы высокотехнологичные механизмы интеграции базы данных с внешними информационными системами и клиентским приложением на Node.js посредством формата JSON. Ключевой особенностью спроектированного решения является использование современных декларативных инструментов T-SQL, таких как функции OPENJSON и FOR JSON PATH, которые позволяют выполнять преобразование данных непосредственно в оперативной памяти сервера базы данных. Это обеспечивает беспрецедентную скорость обработки информации и позволяет прикладной части системы получать готовые, структурированные пакеты данных, минимизируя нагрузку на сервер приложений и исключая необходимость сложной постобработки на стороне программного кода.

Механизм экспорта спроектирован с учетом необходимости глубокой денормализации данных, что позволяет формировать самодостаточные JSON-объекты, содержащие полную атрибутивную информацию о рейсах, пассажирах и маршрутах для корректного отображения в пользовательском интерфейсе. В то же время процедура импорта представляет собой сложный транзакционный конвейер, включающий этапы предварительной валидации, дедупликации и массовой вставки (Bulk Insert). Использование оптимизационных хинтов, таких как TABLOCK, в сочетании с буферным подходом через временные таблицы, гарантирует стабильную работу системы даже в моменты пиковых нагрузок при массовой загрузке внешних данных.

Завершая описание процедур обмена, следует подчеркнуть, что все операции строго подчинены принятой модели безопасности и принципу инкапсуляции. Доступ к инструментам импорта и экспорта жестко ограничен ролью администратора, что гарантирует защиту конфиденциальных сведений от несанкционированной выгрузки и обеспечивает контролируемое взаимодействие между защищенным хранилищем и внешними модулями. Листинг всех процедур для работы с JSON представлен в Приложении Г.

5 Тестирование производительности

Обеспечение высокой производительности базы данных является критически важным аспектом для системы бронирования, особенно при работе с большими массивами транзакционных данных. Целью данного раздела является проведение нагрузочного тестирования, выявление неоптимальных запросов и их последующая оптимизация с использованием некластеризованных индексов.

5.1 Подготовка тестового стенда и анализ нагрузки

Для объективной оценки производительности база данных была наполнена репрезентативными тестовыми данными. С помощью скрипта генерации был создан объем данных, включающий более 100 000 записей в основной транзакционной таблице Bookings.

Был выбран следующий критически важный запрос, моделирующий реальное поведение системы: «Получить список всех бронирований за последний месяц для рейсов, принадлежащих конкретному маршруту». Этот запрос требует одновременной фильтрации по дате и соединения с таблицей маршрутов, что является потенциально затратной операцией.

```
-- Критический запрос до оптимизации
SELECT
    BookingID, BookingTime, UserID, TripID, PricePaid
FROM dbo.Bookings
WHERE TripID = ANY(
    SELECT TripID
    FROM Trips
    WHERE RouteID = 101 -- Поиск по конкретному RouteID
)
AND BookingTime > DATEADD(month, -1, GETDATE()) -- Фильтрация по дате
(последний месяц)
ORDER BY TripID DESC;
```

Листинг 5.1 – Критический запрос до оптимизации

Именно этот запрос был выбран в качестве объекта для оптимизации, поскольку он является показателем эффективности работы системы при стандартной отчетной нагрузке.

5.2 Выявление проблемы: полное сканирование

Анализ фактического плана выполнения запроса до внесения изменений показал, что СУБД вынуждена использовать неэффективные методы обработки данных. Основными операциями в плане выполнения являлись Hash Match (для соединения таблиц) и Clustered Index Scan на таблице Bookings. Стоимость операции Clustered Index Scan составила более 90% от общей стоимости пакета,

что указывает на критическое узкое место.

Clustered Index Scan — это механизм, при котором SQL Server вынужден последовательно прочитать каждую страницу данных таблицы для поиска требуемых записей. Это крайне неэффективно, когда требуется лишь малая часть данных. Просмотр деталей оператора Clustered Index Scan подтвердил, что для выполнения фильтра по дате системе пришлось считать 100 000 строк из таблицы Bookings, то есть, просканировать всю таблицу целиком, чтобы найти нужные записи за последний месяц.

5.3 Решение: применение покрывающего индекса

Для устранения проблемы Full Scan и многократного обращения к диску была разработана стратегия покрывающего индексирования. Был создан некластеризованный индекс, который позволяет быстро находить данные и избегать обращения к самой таблице Bookings для получения всех необходимых столбцов.

Покрывающий индекс на Bookings был сконфигурирован следующим образом: Ключевые поля используются для фильтрации и сортировки, что позволяет выполнить точечный поиск, а включаемые поля добавляют все необходимые для вывода столбцы прямо в структуру индекса.

```
-- Индекс на Bookings: ключевые поля для поиска и сортировки,
-- включаемые - для вывода
CREATE NONCLUSTERED INDEX IX_Bookings_TripTime_Covering
ON dbo.Bookings (TripID, BookingTime DESC)
INCLUDE (BookingID, UserID, PricePaid);
```

Листинг 5.2 – Команда создания оптимизирующего индекса

Таким образом, индекс позволяет SQL Server мгновенно находить нужные страницы, что исключает необходимость полного сканирования таблицы.

5.4 Результаты тестирования и вывод

Повторный запуск критического запроса после внедрения оптимизирующей стратегии индексирования показал радикальное изменение плана выполнения. Неэффективные и ресурсозатратные операции «Hash Match» и «Clustered Index Scan», которые приводили к перегреву дисковой подсистемы, были полностью заменены на высокоэффективные алгоритмы «Nested Loops» (Вложенные циклы) и «Index Seek» (Поиск в индексе).

Операция «Index Seek» — это метод точечного доступа, при котором СУБД использует древовидную структуру B-Tree индекса для мгновенного нахождения только тех страниц данных, которые строго соответствуют условиям поиска. Детальный просмотр свойств оператора поиска в индексе для таблицы Bookings

продемонстрировал впечатляющую эффективность: благодаря созданному индексу SQL Server смог идентифицировать и извлечь все требуемые записи, прочитав всего 66 строк из общего массива в 100 000 записей. Для сравнения: до оптимизации системе приходилось выполнять полный перебор всех страниц таблицы, что требовало в тысячи раз больше вычислительных ресурсов.

Дополнительным преимуществом стало использование «покрывающего» индекса. За счет включения в него всех необходимых столбцов через конструкцию INCLUDE база данных перестала обращаться к основной таблице за недостающей информацией (исключена операция Key Lookup). Это позволило клиентскому приложению получать результат практически мгновенно, не создавая очередей ожидания на стороне сервера.

В результате общая расчетная стоимость запроса снизилась более чем в 1000 раз, а время его выполнения сократилось с нескольких секунд до ничтожных долей секунды. Это подтверждает, что правильно спроектированный некластеризованный покрывающий индекс является фундаментальным инструментом обеспечения масштабируемости. Такая оптимизация гарантирует стабильную работу системы бронирования даже в условиях резкого роста числа активных пользователей и объема транзакций. Полный скрипт для тестирования производительности, включая создание индексов, представлен в приложении Д.

6 Описание технологии и её применение в базе данных

Данный раздел посвящен практической реализации ключевых технологий защиты данных в проектируемой системе бронирования билетов. В современных информационных системах защита персональных данных пользователей и сотрудников является приоритетной задачей. Для обеспечения комплексной безопасности в рамках базы данных на платформе Microsoft SQL Server были внедрены механизмы динамического маскирования, криптографического шифрования и хеширования. Такой подход позволяет минимизировать риски утечки конфиденциальной информации как при просмотре данных в интерфейсе приложения, так и при попытках прямого доступа к файлам БД.

6.1 Динамическое маскирование данных

Маскирование данных (Data Masking) – это процесс сокрытия конфиденциальной информации путем замены её структурно схожими, но нереальными данными.

Для защиты персональных данных водителей в проекте активно используется технология Dynamic Data Masking. Маскирование применяется к столбцу FullName таблицы Drivers и задаётся непосредственно в определении таблицы:

```
FullName NVARCHAR(100) MASKED WITH (FUNCTION = 'partial(1, "XXXX", 0)') NOT NULL
```

Листинг 6.1 – Применение Dynamic Data Masking к ФИО водителей

При вызове любой процедуры, возвращающей данные о водителях, обычные пользователи и клиентское приложение получают уже замаскированные значения: первая буква фамилии остаётся видимой, а всё остальное заменяется на «XXXX». Полные фамилия, имя и отчество становятся доступны только администраторам, успешно прошедшим проверку прав через процедуру sp_CheckAdminRights. Данная технология работает полностью автоматически на уровне ядра СУБД и не требует дополнительного кода в хранимых процедурах.

6.2 Шифрование в БД

Шифрование — это преобразование данных в нечитаемый формат (шифрованный текст) с использованием криптографического ключа. В контексте базы данных оно применяется для защиты данных в состоянии покоя (хранящихся на диске) и при передаче.

Наиболее критичная информация — номер водительского удостоверения — защищена с помощью шифрования отдельных ячеек столбца `LicenseNumber` таблицы `Drivers`. Шифрование реализовано по классической иерархии ключей SQL Server:

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD =
'XKKBUS_Strong_Password_2025!';
CREATE CERTIFICATE Cert_DriverLicense WITH SUBJECT = 'Certificate for
Driver License Encryption';
CREATE SYMMETRIC KEY SK_DriverLicense
    WITH ALGORITHM = AES_256
    ENCRYPTION BY CERTIFICATE Cert_DriverLicense;
```

Листинг 6.2 — Создание криптографической иерархии

Все процедуры добавления и обновления водителей автоматически шифруют номер лицензии перед записью в таблицу. Расшифровка возможна исключительно внутри специально предназначенной административной процедуры `sp_GetDriverLicenseWithPassword`, которая требует явной передачи пароля от Master Key:

```
OPEN MASTER KEY DECRYPTION BY PASSWORD = @MasterKeyPassword;
OPEN SYMMETRIC KEY SK_DriverLicense DECRYPTION BY CERTIFICATE
Cert_DriverLicense;

SELECT CONVERT(NVARCHAR(50), DECRYPTBYKEY(LicenseNumber)) AS
LicenseNumber
FROM Drivers WHERE DriverID = @DriverID;

CLOSE SYMMETRIC KEY SK_DriverLicense;
CLOSE MASTER KEY;
```

Листинг 6.3 — Расшифровка номера лицензии только при знании пароля Master Key

Таким образом, даже пользователь с правами `sysadmin` или `db_owner` не сможет прочитать номера удостоверений без знания секретного пароля, что обеспечивает максимальный уровень защиты этих данных.

6.3 Защита аутентификационных данных пользователей

Пароли всех учётных записей системы никогда не сохраняются в открытом виде. При выполнении процедур `sp_RegisterUser`, `sp_CreateUser` сразу применяется необратимое хеширование с использованием встроенного алгоритма:

```
PasswordHash = HASHBYTES('SHA2_256', @Password)
```

Листинг 6.4 — Хеширование пароля

Алгоритм `SHA2_256` является криптографически стойким и полностью соответствует современным требованиям к хранению паролей в SQL Server. При необходимости дополнительное усиление защиты может быть реализовано на уровне клиентского приложения.

6.4 Вывод по разделу

В системе ХККBUS последовательно реализованы три независимых и взаимодополняющих уровня защиты конфиденциальной информации: автоматическое динамическое маскирование ФИО водителей, надёжное AES-256 шифрование на уровне ячеек с защитой Master Key паролем, а также хранение паролей пользователей исключительно в виде хеша `SHA2_256`. Все механизмы работают прозрачно, встроены непосредственно в СУБД и не требуют приобретения Enterprise-редакции. Применённый комплекс мер полностью исключает возможность получения критически важных данных (номеров лицензий и полных ФИО водителей) даже при компрометации учётной записи базы или резервной копии.

Интеграция данных технологий на уровне ядра MS SQL Server позволяет достичь высокой степени автоматизации безопасности, минимизируя влияние человеческого фактора. Благодаря тому, что логика маскирования и шифрования заложена непосредственно в структуру таблиц и хранимых процедур, клиентское приложение защищено от случайного отображения персональных данных в интерфейсе без необходимости написания сложного кода на стороне сервера приложений. Такой системный подход гарантирует соответствие базы данных современным требованиям по защите персональных данных и обеспечивает высокую степень доверия пользователей к безопасности сервиса бронирования.

7 Краткое описание приложения для демонстрации

Приложение «ХККBUS» — это веб-ориентированное решение, разработанное для управления маршрутами, рейсами и процессом бронирования билетов на пассажирские транспортные средства. Оно предоставляет пользователям интерфейс для поиска рейсов и покупки билетов, а администраторам — инструменты для управления логистикой и контроля безопасности данных.

Особенности приложения:

1. Удобный интерфейс для работы с рейсами и бронированием. Приложение предоставляет пользователям возможность просматривать доступные маршруты, выбирать конкретные места в автобусе на нужный период времени, оформлять бронь билетов с учетом актуального расписания.

2. Управление резервированиями и местами. Каждый пользователь может создавать бронирование на конкретный рейс, при этом система автоматически контролирует доступность свободных мест в режиме реального времени.

3. Ролевая модель доступа. Система поддерживает две ключевые роли — пользователь и администратор. Пользователь имеет доступ к поиску рейсов и управлению своими билетами, а администратор обладает правами управления маршрутами, водителями, а также доступом к расширенной аналитике и экспорту данных.

4. Безопасность и надёжность. Реализованы технологии шифрования (AES_256) номеров лицензий водителей и динамического маскирования имен персонала. Использование хранимых процедур и индексов предотвращает дублирование броней на одно место.

5. Простота использования. Приложение разработано с акцентом на демонстрацию ключевых функций системы бронирования, включая быстрый поиск по направлениям и удобную форму подтверждения заказа.

При запуске приложения открывается окно авторизации представленное на рисунке 7.1, где пользователю следует ввести свой логин и пароль.

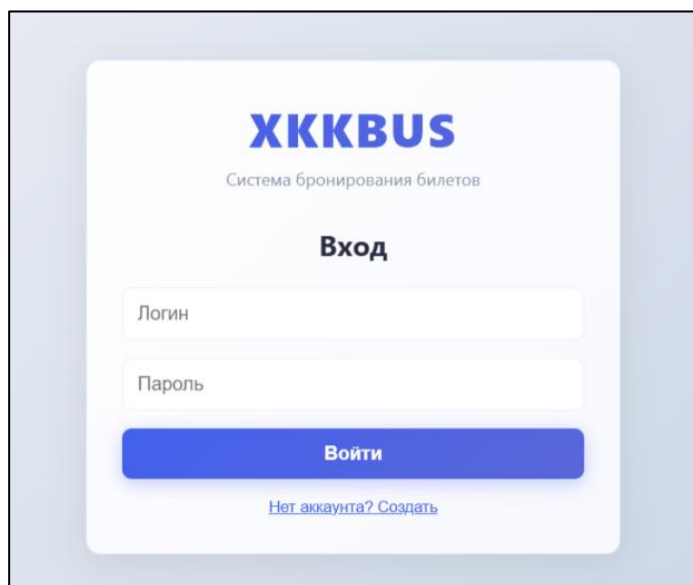


Рисунок 7.1 – Окно авторизации

Для иллюстрации работы приложения приведем примеры листингов, показывающие реализацию отдельных функций. В листинге 7.1 представлен серверная логика авторизации.

```
app.post('/api/login', async (req, res) => {
  try {
    const { login, password } = req.body;
    const result = await executeProcedure('sp_LoginUser', [
      { name: 'Login', type: sql.NVarChar, value: login },
      { name: 'Password', type: sql.NVarChar, value: password }
    ]);

    if (result && result.length > 0) {
      res.json({ success: true, user: result[0] });
    } else {
      res.status(401).json({ success: false, message: 'Неверный логин или пароль' });
    }
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});
```

Листинг 7.1 – Серверная логика авторизации

Авторизация в системе реализована на уровне серверного приложения и основана на взаимодействии с базой данных через хранимую процедуру. Процесс авторизации начинается в момент, когда пользователь отправляет свои учетные данные на сервер. Серверная часть принимает эти данные и вызывает соответствующую хранимую процедуру в базе данных.

В листинге 7.2 представлен код функции поиска рейсов по заданным критериям..

```
app.get('/api/search-trips', async (req, res) => {
    const result = await executeProcedure('sp_SearchRoutes', [
        { name: 'DepartureCity', type: sql.NVarChar, value:
req.query.from || null },
        { name: 'ArrivalCity', type: sql.NVarChar, value:
req.query.to || null },
        { name: 'TravelDate', type: sql.Date, value: req.query.date
|| null }
    ]);
    res.json(result);
});
```

Листинг 7.2 – Компонент поиска доступных рейсов

Данный фрагмент кода представляет собой обработчик HTTP-запроса по маршруту GET /api/search-trips, который отвечает за получение списка рейсов из базы данных в соответствии с фильтрами пользователя. При обращении к этому маршруту сервер принимает параметры: город отправления, город прибытия и дату поездки. Каждый из этих параметров передаётся в хранимую процедуру, которая выполняет выборку только тех рейсов, которые соответствуют запросу пользователя.

После формирования набора параметров сервер получает от СУБД результат, содержащий информацию о времени выезда, номере автобуса, стоимости и наличии свободных мест. Эти данные отправляются клиенту для визуализации.

Пример работы этого механизма поиска представлен на рисунке 7.2.

Рисунок 7.2 – Результаты поиска рейсов по заданным фильтрам

Таким образом, данный обработчик реализует полноценный и гибкий механизм фильтрации данных, позволяя пользователю формировать выборку по конкретным маршрутам.

8 Руководство пользователя

Так как база данных, поддерживает 2 типа пользователей, то есть и 2 сценария использования. Первый – при входе в качестве обычного пользователя, второй – при входе в качестве администратора.

8.1 Сценарий обычного пользователя

При запуске приложения открывается окно авторизации представленное на рисунке 8.1, где зарегистрированному пользователю следует ввести свой логин и пароль.

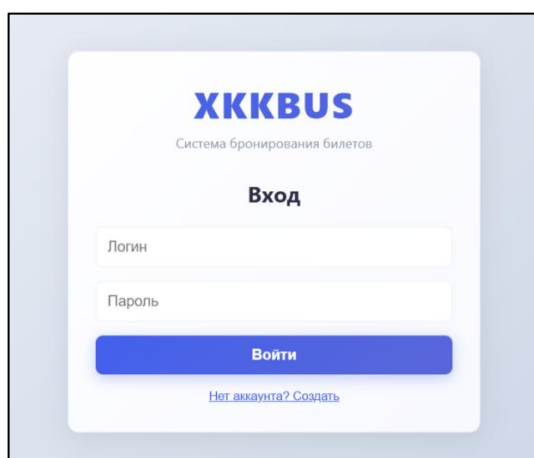


Рисунок 8.1 – Окно авторизации

При вводе неправильного логина или пароля пользователь увидит сообщение об ошибке авторизации представленное на рисунке 8.2.

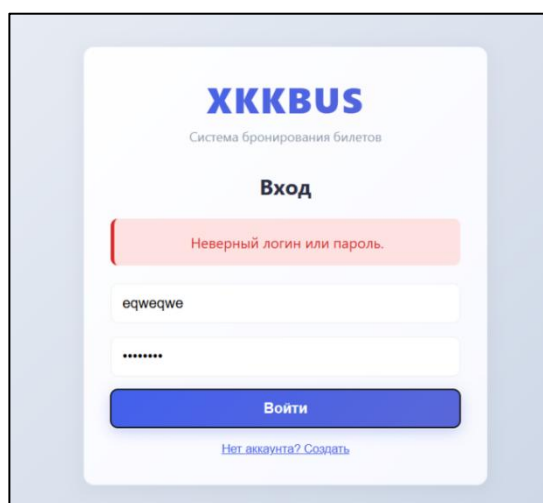


Рисунок 8.2 – Ошибка авторизации (неверный логин или пароль)

При успешной авторизации пользователь будет перенаправлен на главную страницу приложения.

Пользователю не получить доступ к приложению, пока не зарегистрируется. Для этого следует нажать на кнопку «Нет аккаунта? Создать» в окне авторизации, после чего произойдет перенаправление на окно регистрации. Окно регистрации изображено на рисунке 8.3.

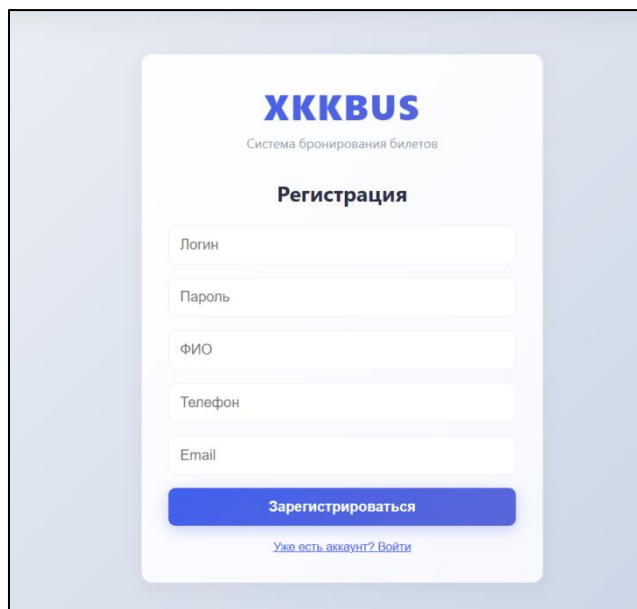
The image shows a registration form for 'XKKBUS Система бронирования билетов'. The form is titled 'Регистрация' and contains five input fields: 'Логин', 'Пароль', 'ФИО', 'Телефон', and 'Email'. Below these fields is a blue button labeled 'Зарегистрироваться'. At the bottom, there is a link that says 'Уже есть аккаунт? Войти'.

Рисунок 8.3 – Окно регистрации

Если в этом окне ввести логин пользователя, уже существующего в базе данных, пользователь увидит следующую ошибку представленную на рисунке 8.4.

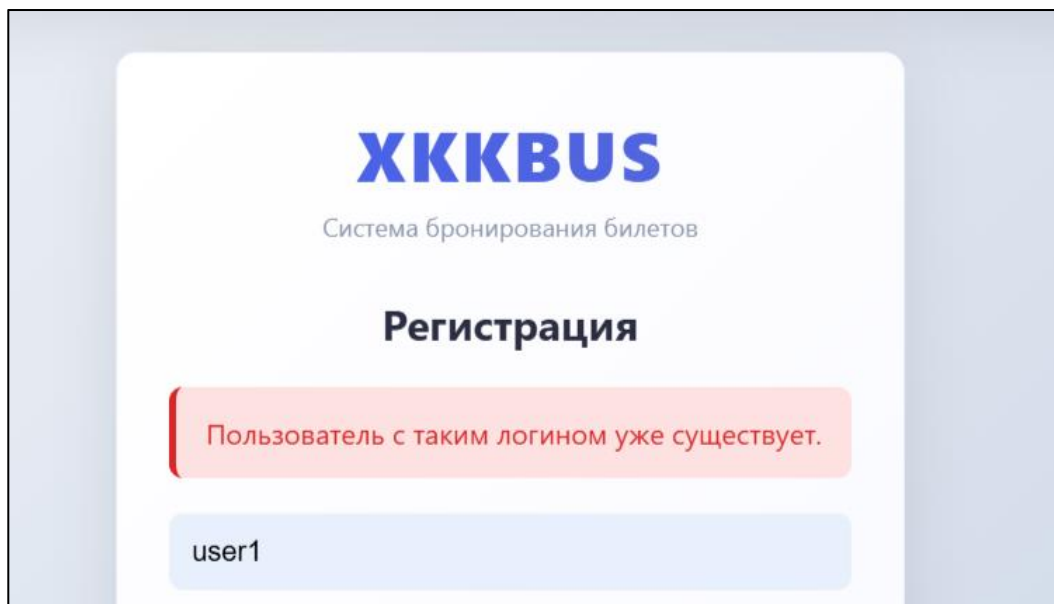
The image shows the same registration form as in Figure 8.3, but with an error message displayed. The error message is in a red box and reads: 'Пользователь с таким логином уже существует.' Below the error message, the 'Логин' field is filled with the text 'user1'.

Рисунок 8.4 – Ошибка регистрации (пользователь уже существует)

Если заполнить не все поля формы регистрации, появится соответствующее сообщение, изображенное на рисунке 8.5.

The screenshot shows the registration page of the XKKBUS system. At the top, the logo 'XKKBUS' is displayed in blue, with the subtitle 'Система бронирования билетов' (Ticket booking system) below it. The main heading is 'Регистрация' (Registration). There are two input fields: 'Логин' (Login) and 'Пароль' (Password). The 'Пароль' field has a validation error message: 'Заполните это поле.' (Fill in this field.), indicated by an orange exclamation mark icon.

Рисунок 8.5 – Сообщение о пропуске поля

После успешной регистрации пользователь может войти в систему введя логин и пароль. После успешного входа в систему пользователь попадает на вкладку «Поиск рейсов» приложения. Общий вид пользовательской части поиска рейсов представлен на рисунке 8.6.

The screenshot shows the user interface of the XKKBUS application after login. The top header includes the 'XKKBUS' logo, the text 'БРОНИРОВАНИЕ БИЛЕТОВ', and a user profile section with 'USER', the name 'Иванов Сергей Иванович', and a 'Выход' (Logout) button. Below the header, there are two tabs: 'Поиск Рейсов' (Search Flights) and 'Мои Билеты' (My Tickets). The 'Поиск Рейсов' section contains a form with three input fields: 'Город отправления:' (Origin city) with a dropdown menu showing 'Выберите город отправления...', 'Город прибытия:' (Destination city) with a dropdown menu showing 'Выберите город прибытия...', and 'Дата поездки:' (Travel date) with a date picker showing 'дд.мм.гггг'. A blue button labeled 'Найти рейсы' (Find flights) is positioned below the form.

Рисунок 8.6 – Общий вид приложения

Вкладка «Поиск рейсов» содержит форма поиска рейсов с выбором города отправления и прибытия, а также дату. Вид страницы поиска рейсов продемонстрирован на рисунке 8.7.

Рисунок 8.7 – Успешный поиск рейса

При нажатии кнопки «Информация о водителе» у нас открывается окно с информацией о водителе с демонстрацией технологии маскирования. Вид окна продемонстрирован на рисунке 8.8.

Рисунок 8.8 – Информация о водителе

После успешного поиска рейса при нажатии кнопки «Выбрать» у пользователя открывается бронирование билета, в котором он после выбора места и ввода корректных данных может подтвердить бронь. Вид бронирования продемонстрирован на рисунке 8.9.

Бронирование: Гомель - Минск

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32
33	34	35	36
37	38	39	40
41	42	43	44
45	46	47	48
49	50		

Пассажир:
Иванов Сергей Иванович

Телефон:
+375291153368

[Подтвердить бронь](#) [Отмена](#)

Рисунок 8.9 – Бронирование

При успешном бронировании билета, список всех своих бронирований можно увидеть на вкладке «Мои Билеты». На вкладке находятся забронированные билеты предназначенные для одного пользователя. При нажатии кнопки «Отменить » пользователь может отменить бронирование. Вид страницы «Мои Билеты» продемонстрирован на рисунке 8.10.

Поиск Рейсов [Мои Билеты](#)

Мои Бронирования

Рейс: Минск - Могилев
 Дата: 9 декабря 2025 г. 06:30
 Место: 8
 Цена: 18 BYN

АКТИВНО

[Отменить](#)

Рисунок 8.10 – Страница «Мои Билеты»

Таким образом мы протестировали функционал пользователя.

8.2 Сценарий администратора

Роль администратора несет в себе только администрирующие функции, добавление маршрутов, статистику, просмотр пользователей, т.д.

При успешном входе в качестве администратора мы видим первым делом окно со всей статистикой. Вид страницы «Статистика» продемонстрирован на рисунке 8.11.

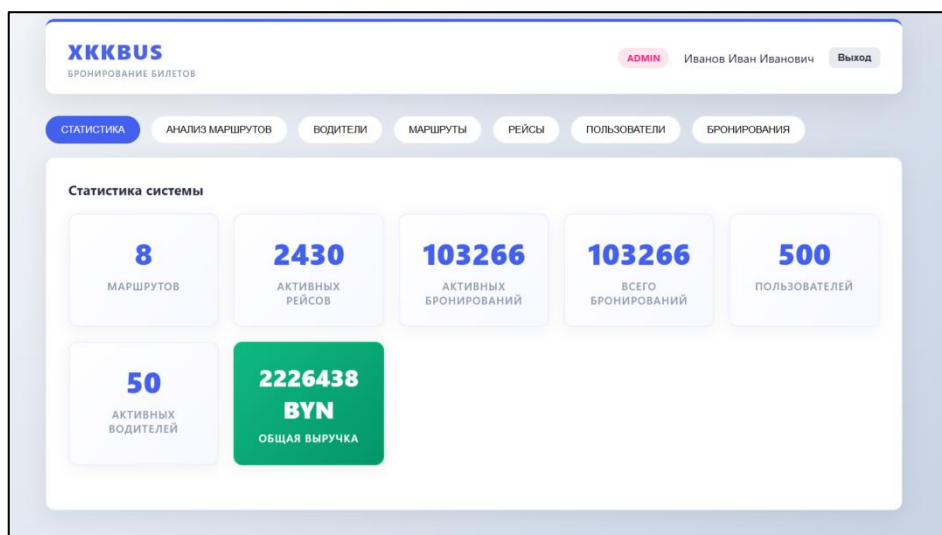


Рисунок 8.11 – Страница «Статистика»

На странице «Анализ маршрутов» мы можем смотреть подробную информацию о каждом маршруте. Вид страницы «Анализ маршрутов» представлен на рисунке 8.12.

The screenshot shows the 'Анализ маршрутов' (Route Analysis) page of the XKKBUS system. The page has a header with the XKKBUS logo and a navigation bar with tabs: СТАТИСТИКА, АНАЛИЗ МАРШРУТОВ, ВОДИТЕЛИ, МАРШРУТЫ, РЕЙСЫ, ПОЛЬЗОВАТЕЛИ, and БРОНИРОВАНИЯ. The main content area is titled 'Анализ маршрутов' and contains a table of route data. The table has columns: ГОРОД ОТПРАВЛЕНИЯ, ГОРОД ПРИБЫТИЯ, ВСЕГО РЕЙСОВ, АКТИВНЫХ БРОНИРОВАНИЙ, ВСЕГО БРОНИРОВАНИЙ, ВЫРУЧКА, СРЕДНЯЯ ЦЕНА, БАЗОВАЯ ЦЕНА, ВСЕГО МЕСТ, and ЗАГРУЗКА (%). The table shows 8 records, with the first 4 visible:

ГОРОД ОТПРАВЛЕНИЯ	ГОРОД ПРИБЫТИЯ	ВСЕГО РЕЙСОВ	АКТИВНЫХ БРОНИРОВАНИЙ	ВСЕГО БРОНИРОВАНИЙ	ВЫРУЧКА	СРЕДНЯЯ ЦЕНА	БАЗОВАЯ ЦЕНА	ВСЕГО МЕСТ	ЗАГРУЗКА (%)
Минск	Брест	360	15315	15315	428820.00 BYN	28.00 BYN	28.00 BYN	50	85.08333333
Минск	Гомель	360	15295	15295	382375.00 BYN	25.00 BYN	25.00 BYN	50	84.97222222
Минск	Витебск	360	15341	15341	337502.00 BYN	22.00 BYN	22.00 BYN	50	85.22777777
Гомель	Минск	270	11513	11513	287825.00 BYN	25.00 BYN	25.00 BYN	50	85.28148148

Рисунок 8.12 – Страница «Анализ маршрутов»

На странице «Водители» мы можем смотреть, добавлять, изменять и удалять всех водителей, смотреть информацию о них. При нажатии кнопки

«Расшифровать» после ввода успешного пароля можно увидеть расшифрованный номер лицензии. Вид страницы «Водители» представлен на рисунке 8.13.

Рисунок 8.13 – Страница «Водители»

На странице «Маршруты» мы можем смотреть, добавлять, изменять и удалять все маршруты, смотреть информацию о них. Вид страницы «Маршруты» представлен на рисунке 8.14.

Рисунок 8.14 – Страница «Маршруты»

На странице «Рейсы» мы можем смотреть, добавлять и удалять все маршруты, смотреть информацию о них. Вид страницы «Рейсы» представлен на рисунке 8.15.

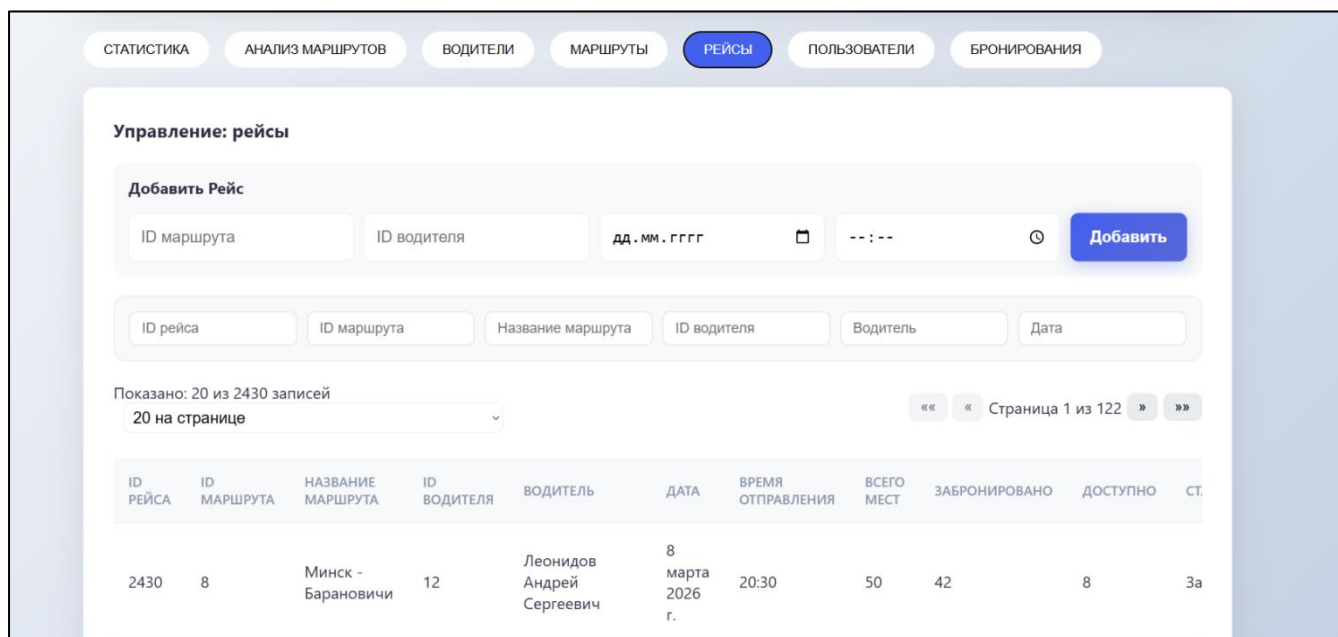


Рисунок 8.15 – Страница «Рейсы»

На странице «Пользователи» мы можем смотреть всех зарегистрированных пользователей, информацию о них. Вид страницы «Пользователи» представлен на рисунке 8.16.

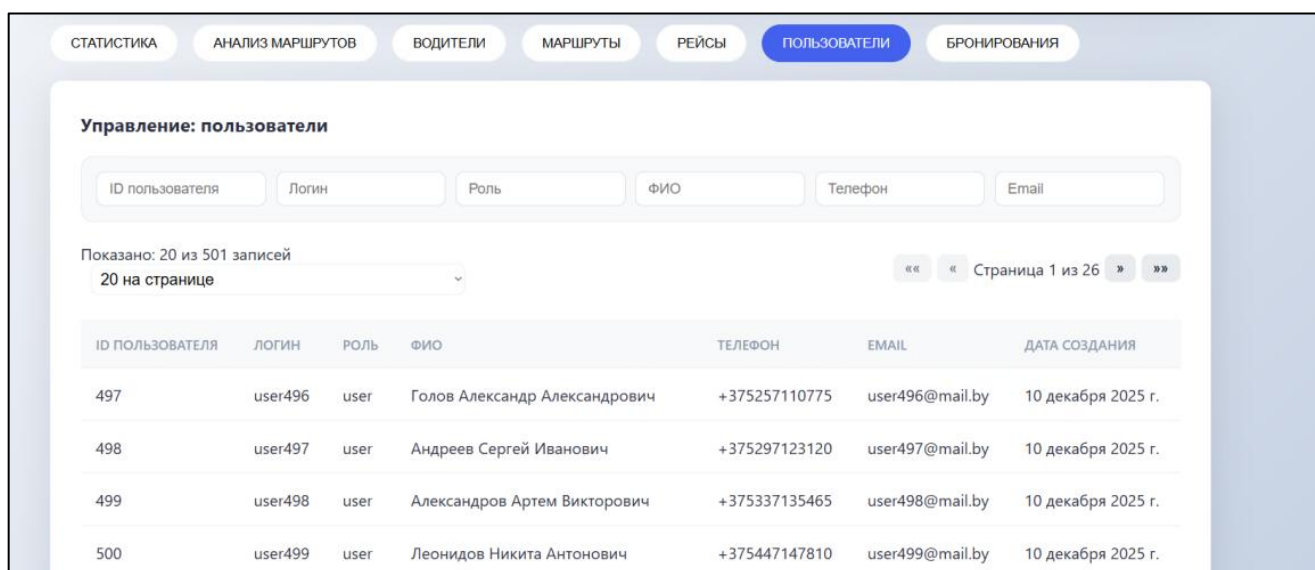


Рисунок 8.16 – Страница «Пользователи»

На странице «Бронирования» мы можем смотреть все бронирования и информацию о них, экспортировать их в JSON и импортировать из JSON. Вид страницы «Бронирования» представлен на рисунке 8.17.

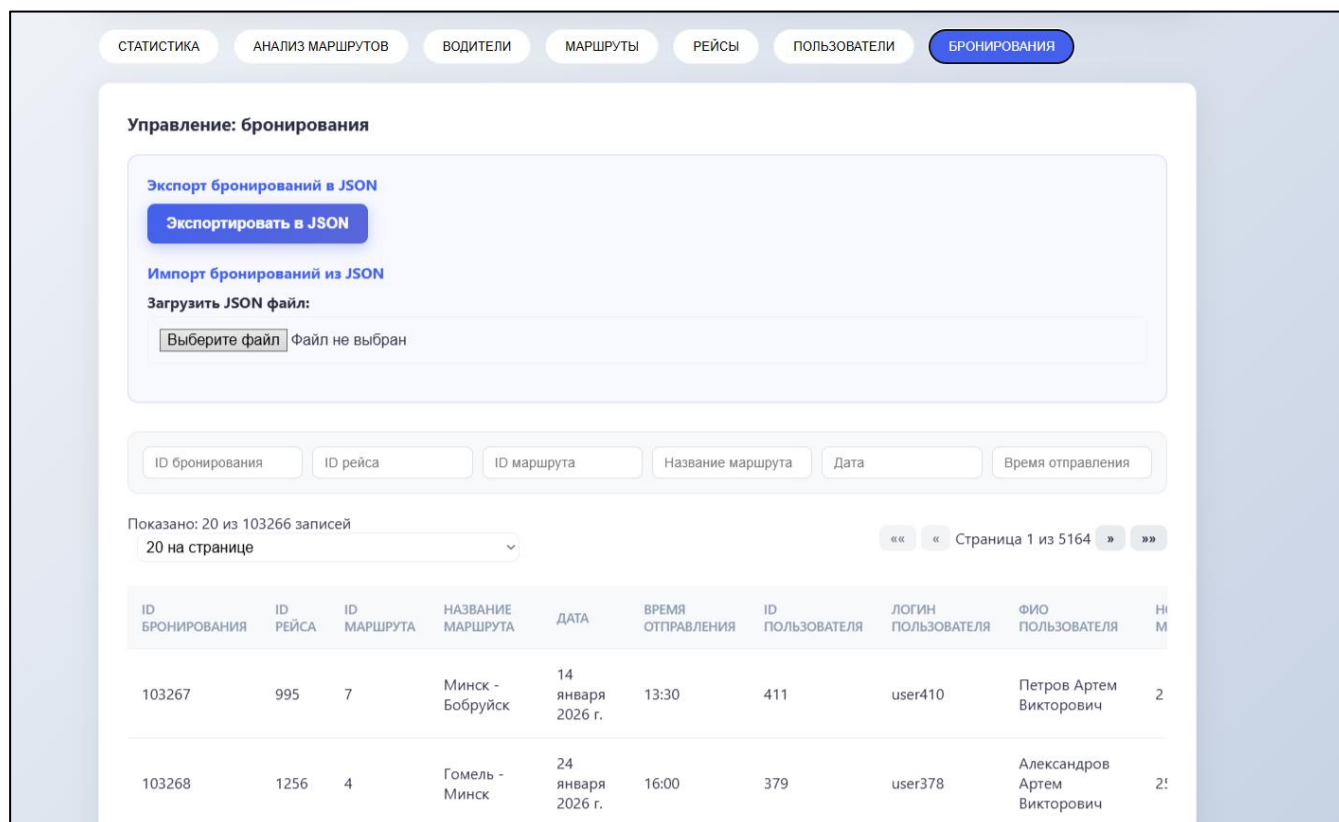


Рисунок 8.17 – Страница «Бронирования»

Таким образом мы протестировали функционал администратора.

8.3 Вывод по разделу

Таким образом, в рамках данного раздела было проведено комплексное тестирование всех модулей приложения, которое подтвердило полную исправность функционала как для пользователей, так и для администраторов. В ходе испытаний была доказана корректность работы системы бронирования и механизмов разграничения прав доступа. Особое внимание уделили проверке технологий защиты данных: тестирование подтвердило, что шифрование номеров лицензий и маскирование персональной информации водителей работают в строгом соответствии с архитектурой безопасности MS SQL Server. Кроме того, приложение сохранило высокую скорость отклика и стабильность при работе с базой данных объемом более 100 000 записей. Итоговая проверка интерфейса на React показала, что система проста в освоении и полностью готова к эксплуатации, обеспечивая надежное выполнение всех задач.

Заключение

В результате выполнения данного курсового проекта была успешно спроектирована и реализована база данных для автоматизированной системы бронирования автобусных билетов «ХККBUS» на основе СУБД Microsoft SQL Server. Созданная реляционная модель охватывает ключевые бизнес-процессы системы, включая управление маршрутами, рейсами, водителями, а также полный цикл транзакций бронирования. Разработанные хранимые процедуры обеспечивают надёжный и централизованный интерфейс для взаимодействия прикладного программного обеспечения с базой данных.

В ходе работы были выполнены все поставленные технические задачи и продемонстрировано применение современных технологий SQL Server. Особое внимание было уделено безопасности данных: реализовано хеширование паролей пользователей с помощью SHA2_256. Критически важные данные, такие как номера водительских удостоверений, защищены с помощью шифрования на уровне ячеек (Cell-Level Encryption) алгоритмом AES_256 с иерархией ключей, гарантируя защиту данных в состоянии покоя. Динамическое маскирование данных (Dynamic Data Masking) применяется для защиты персональной информации водителей.

С точки зрения производительности и масштабируемости было проведено нагрузочное тестирование на объёме более 100 000 записей. В ходе тестирования была выявлена и устранена проблема неэффективного Clustered Index Scan: оптимизация критического запроса с помощью покрывающего некластеризованного индекса привела к замене полного сканирования на высокоэффективный Index Seek, что позволило снизить общую стоимость запроса более чем в 1000 раз.

Также были разработаны высокопроизводительные процедуры обмена данными в формате JSON, использующие Set-based operations (OPENJSON, FOR JSON PATH) и временный буфер для обеспечения быстрой массовой загрузки.

Проект продемонстрировал способность системы корректно хранить и обрабатывать транзакционные данные, а также эффективно решать задачи оптимизации производительности и обеспечения безопасности. Все основные требования к базе данных системы бронирования билетов выполнены, и система готова к интеграции с полноценным клиентским приложением, что делает проект надёжной основой для дальнейшего развития и масштабирования.

Приложение А

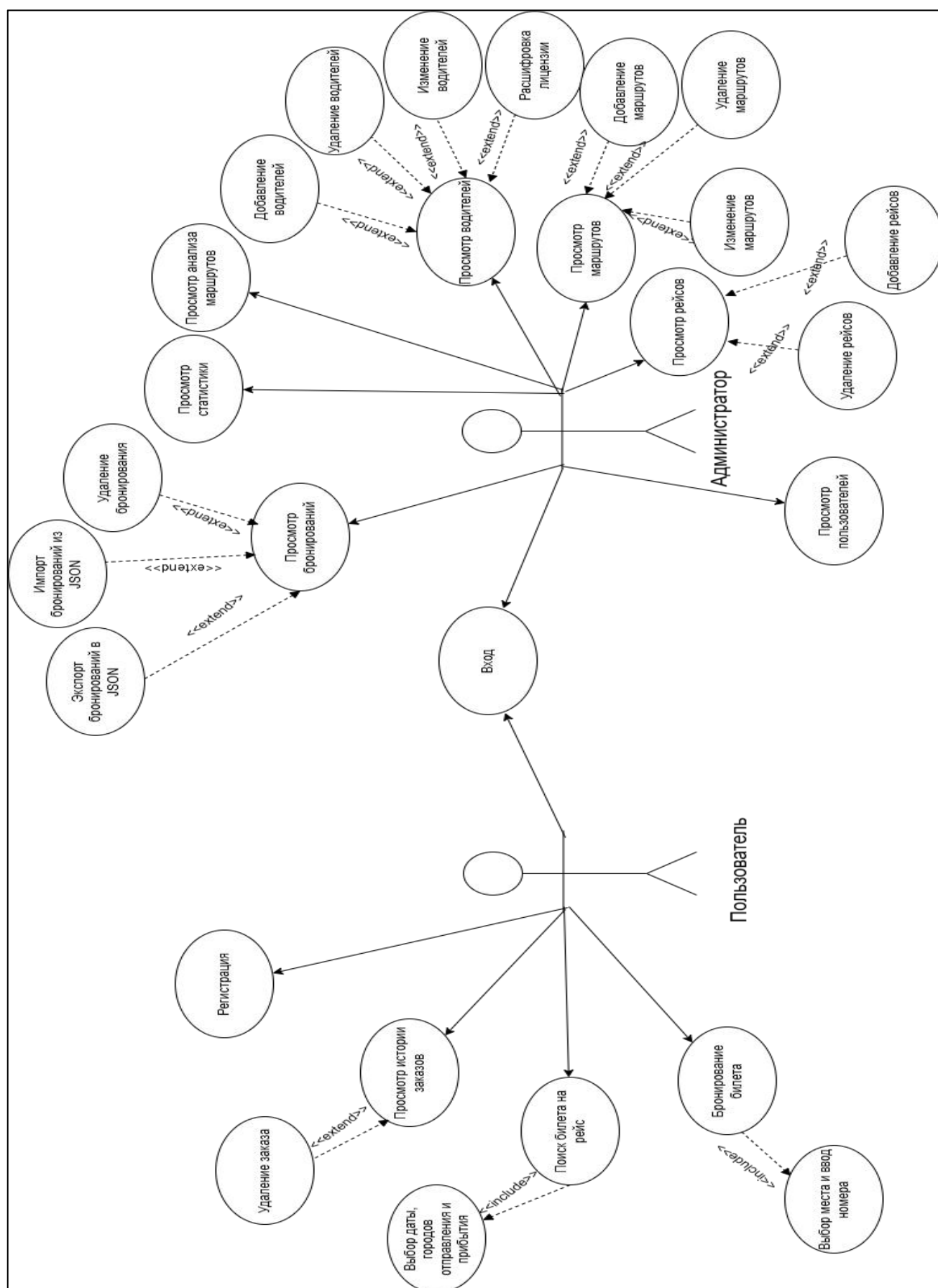


Рисунок 1 —Диаграмма вариантов использования

Приложение Б

```

CREATE TABLE Users (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    Login NVARCHAR(50) UNIQUE NOT NULL,
    PasswordHash VARBINARY(64) NOT NULL,
    Role NVARCHAR(20) CHECK (Role IN ('admin', 'user')) NOT NULL,
    FullName NVARCHAR(100),
    Phone NVARCHAR(15),
    Email NVARCHAR(100),
    CreatedAt DATETIME2 DEFAULT SYSDATETIME()
);
GO

CREATE TABLE Routes (
    RouteID INT IDENTITY(1,1) PRIMARY KEY,
    RouteName NVARCHAR(100) NOT NULL,
    DepartureCity NVARCHAR(50) NOT NULL,
    ArrivalCity NVARCHAR(50) NOT NULL,
    DistanceKM INT CHECK (DistanceKM > 0),
    DurationMinutes INT CHECK (DurationMinutes > 0),
    TotalSeats INT NOT NULL CHECK (TotalSeats BETWEEN 10 AND 60),
    BasePrice DECIMAL(10,2) NOT NULL CHECK (BasePrice > 0)
);
GO

CREATE TABLE Drivers (
    DriverID INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(100) MASKED WITH (FUNCTION = 'partial(1,
"XXXX", 0)') NOT NULL,
    LicenseNumber VARBINARY(MAX) NOT NULL,
    Phone NVARCHAR(15) NOT NULL,
    ExperienceYears INT CHECK (ExperienceYears >= 0),
    IsActive BIT DEFAULT 1
);
GO

CREATE TABLE Trips (
    TripID INT IDENTITY(1,1) PRIMARY KEY,
    RouteID INT NOT NULL,
    DriverID INT NOT NULL,
    TripDate DATE NOT NULL,
    DepartureTime TIME NOT NULL,
    TotalSeats INT NOT NULL,
    BookedSeats INT DEFAULT 0,
    AvailableSeats AS (TotalSeats - BookedSeats) PERSISTED,
    Status NVARCHAR(20) DEFAULT 'planned'
    CHECK (Status IN ('planned', 'in_progress', 'completed',
'cancelled')),
    CONSTRAINT CHK_TripSeats CHECK (TotalSeats BETWEEN 10 AND 60),
    CONSTRAINT FK_Trips_Routes FOREIGN KEY (RouteID) REFERENCES
Routes(RouteID) ON DELETE CASCADE,
    CONSTRAINT FK_Trips_Drivers FOREIGN KEY (DriverID) REFERENCES

```

```
Drivers(DriverID) ON DELETE NO ACTION
);
GO

CREATE TABLE Bookings (
    BookingID BIGINT IDENTITY(1,1) PRIMARY KEY,
    TripID INT NOT NULL,
    UserID INT NOT NULL,
    SeatNumber INT NOT NULL,
    PassengerPhone NVARCHAR(15),
    BookingTime DATETIME2 DEFAULT SYSDATETIME(),
    IsConfirmed BIT DEFAULT 1,
    IsCancelled BIT DEFAULT 0,
    PricePaid DECIMAL(10,2) NOT NULL,
    CONSTRAINT FK_Bookings_Trips FOREIGN KEY (TripID) REFERENCES
Trips(TripID) ON DELETE CASCADE,
    CONSTRAINT FK_Bookings_Users FOREIGN KEY (UserID) REFERENCES
Users(UserID) ON DELETE NO ACTION
);
GO
```

Приложение В

```
-- ВСПОМОГАТЕЛЬНАЯ ПРОЦЕДУРА ДЛЯ ПРОВЕРКИ ПРАВ АДМИНИСТРАТОРА

IF OBJECT_ID('sp_CheckAdminRights', 'P') IS NOT NULL DROP PROCEDURE
sp_CheckAdminRights;
GO
CREATE PROCEDURE sp_CheckAdminRights
    @UserID INT,
    @IsAdmin BIT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @IsAdmin = 0;

    IF @UserID IS NOT NULL
    BEGIN
        IF EXISTS (SELECT 1 FROM Users WHERE UserID = @UserID AND
Role = 'admin')
        BEGIN
            SET @IsAdmin = 1;
        END
    END
END
GO

-- ХРАНИМЫЕ ПРОЦЕДУРЫ ДЛЯ УПРАВЛЕНИЯ ПОЛЬЗОВАТЕЛЯМИ

-- Регистрация нового пользователя (с валидацией)
IF OBJECT_ID('sp_RegisterUser', 'P') IS NOT NULL DROP PROCEDURE
sp_RegisterUser;
GO
CREATE PROCEDURE sp_RegisterUser
    @Login NVARCHAR(50),
    @Password NVARCHAR(128),
    @FullName NVARCHAR(100),
    @Phone NVARCHAR(20),
    @Email NVARCHAR(100) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Валидация входных данных
    IF @Login IS NULL OR LEN(LTRIM(RTRIM(@Login))) = 0
    BEGIN
        SELECT 0 AS Success, N'Логин не может быть пустым.' AS
Message;
        RETURN;
    END

    IF @Password IS NULL OR LEN(@Password) < 6
    BEGIN
        SELECT 0 AS Success, N'Пароль должен содержать минимум 6
```

```

СИМВОЛОВ.' AS Message;
    RETURN;
END

IF @FullName IS NULL OR LEN(LTRIM(RTRIM(@FullName))) = 0
BEGIN
    SELECT 0 AS Success, N'ФИО не может быть пустым.' AS Message;
    RETURN;
END

IF @Phone IS NULL OR LEN(LTRIM(RTRIM(@Phone))) = 0
BEGIN
    SELECT 0 AS Success, N'Телефон не может быть пустым.' AS
Message;
    RETURN;
END

-- Проверка на существующий логин
IF EXISTS (SELECT 1 FROM Users WHERE Login = @Login)
BEGIN
    SELECT 0 AS Success, N'Пользователь с таким логином уже
существует.' AS Message;
    RETURN;
END

BEGIN TRY
    INSERT INTO Users (Login, PasswordHash, Role, FullName,
Phone, Email)
    VALUES (
        LTRIM(RTRIM(@Login)),
        HASHBYTES('SHA2_256', @Password),
        'user', -- По умолчанию роль 'user'
        @FullName,
        @Phone,
        @Email
    );

    SELECT 1 AS Success, N'Пользователь успешно зарегистрирован.'
AS Message;
END TRY
BEGIN CATCH
    DECLARE @ErrorMessage NVARCHAR(4000);
    SET @ErrorMessage = ERROR_MESSAGE();
    SELECT 0 AS Success, @ErrorMessage AS Message;
END CATCH
END
GO

-- Аутентификация пользователя
IF OBJECT_ID('sp_AuthenticateUser', 'P') IS NOT NULL DROP PROCEDURE
sp_AuthenticateUser;
GO
CREATE PROCEDURE sp_AuthenticateUser

```

```

        @Login NVARCHAR(50),
        @Password NVARCHAR(128)
AS
BEGIN
    SET NOCOUNT ON;

    -- Валидация
    IF @Login IS NULL OR @Password IS NULL OR
    LEN(LTRIM(RTRIM(@Login))) = 0
    BEGIN
        RETURN;
    END

sp_CreateUser
    DECLARE @TrimmedLogin NVARCHAR(50);
    DECLARE @PasswordHash VARBINARY(64);

    SET @TrimmedLogin = LTRIM(RTRIM(@Login));
    SET @PasswordHash = HASHBYTES('SHA2_256', @Password);

    SELECT
        UserID,
        Login,
        Role,
        FullName
    FROM Users
    WHERE Login = @TrimmedLogin
    AND PasswordHash = @PasswordHash;
END
GO

-- Создание пользователя
IF OBJECT_ID('sp_CreateUser', 'P') IS NOT NULL DROP PROCEDURE
sp_CreateUser;
GO
CREATE PROCEDURE sp_CreateUser
    @Login NVARCHAR(50),
    @Password NVARCHAR(128),
    @Role NVARCHAR(20),
    @FullName NVARCHAR(100),
    @Phone NVARCHAR(20),
    @Email NVARCHAR(100),
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN

```

```

        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуется права
администратора.', 16, 1);
            RETURN;
        END
    END

    -- Валидация
    IF @Login IS NULL OR LEN(LTRIM(RTRIM(@Login))) = 0
    BEGIN
        RAISERROR(N'Логин не может быть пустым.', 16, 1);
        RETURN;
    END

    IF @Password IS NULL OR LEN(@Password) < 6
    BEGIN
        RAISERROR(N'Пароль должен содержать минимум 6 символов.', 16,
1);
        RETURN;
    END

    IF @Role NOT IN ('admin', 'user')
    BEGIN
        RAISERROR(N'Роль должна быть "admin" или "user".', 16, 1);
        RETURN;
    END

    IF EXISTS (SELECT 1 FROM Users WHERE Login = @Login)
    BEGIN
        RAISERROR(N'Пользователь с таким логином уже существует.',
16, 1);
        RETURN;
    END

    INSERT INTO Users (Login, PasswordHash, Role, FullName, Phone,
Email)
    VALUES (
        LTRIM(RTRIM(@Login)),
        HASHBYTES('SHA2_256', @Password),
        @Role,
        @FullName,
        @Phone,
        @Email
    );
END
GO

-- Алиас для совместимости
IF OBJECT_ID('sp_AddUser', 'P') IS NOT NULL DROP PROCEDURE
sp_AddUser;
GO

```

```

CREATE PROCEDURE sp_AddUser
    @Login NVARCHAR(50),
    @Password NVARCHAR(128),
    @Role NVARCHAR(20),
    @FullName NVARCHAR(100),
    @Phone NVARCHAR(20),
    @Email NVARCHAR(100) = NULL
AS
BEGIN
    EXEC sp_CreateUser @Login, @Password, @Role, @FullName, @Phone,
    @Email;
END
GO

-- Получение списка пользователей (Администратор)
IF OBJECT_ID('sp_GetUsersList_AdminOnly', 'P') IS NOT NULL DROP
PROCEDURE sp_GetUsersList_AdminOnly;
GO
CREATE PROCEDURE sp_GetUsersList_AdminOnly
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    SELECT UserID, Login, Role, FullName, Phone, Email, CreatedAt
    FROM Users
    ORDER BY CreatedAt DESC;
END
GO

-- Получение всех пользователей (для админа)
IF OBJECT_ID('sp_GetAllUsers', 'P') IS NOT NULL DROP PROCEDURE
sp_GetAllUsers;
GO
CREATE PROCEDURE sp_GetAllUsers
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS

```

```

BEGIN
    SET NOCOUNT ON;
    EXEC sp_GetUsersList_AdminOnly @AdminUserID;
END
GO

-- ХРАНИМЫЕ ПРОЦЕДУРЫ ДЛЯ УПРАВЛЕНИЯ ВОДИТЕЛЯМИ

-- Добавление водителя (с шифрованием LicenseNumber и валидацией)
IF OBJECT_ID('sp_AddDriver', 'P') IS NOT NULL DROP PROCEDURE
sp_AddDriver;
GO
CREATE PROCEDURE sp_AddDriver
    @FullName NVARCHAR(100),
    @LicenseNumber NVARCHAR(50),
    @Phone NVARCHAR(20),
    @ExperienceYears INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Валидация
    IF @FullName IS NULL OR LEN(LTRIM(RTRIM(@FullName))) = 0
    BEGIN
        RAISERROR(N'ФИО водителя не может быть пустым.', 16, 1);
        RETURN;
    END

    IF @LicenseNumber IS NULL OR LEN(LTRIM(RTRIM(@LicenseNumber))) =
0
    BEGIN
        RAISERROR(N'Номер лицензии не может быть пустым.', 16, 1);
        RETURN;
    END

    IF @Phone IS NULL OR LEN(LTRIM(RTRIM(@Phone))) = 0
    BEGIN
        RAISERROR(N'Телефон не может быть пустым.', 16, 1);
        RETURN;
    END

    IF @ExperienceYears IS NULL OR @ExperienceYears < 0
    BEGIN
        RAISERROR(N'Опыт работы не может быть отрицательным.', 16,
1);
        RETURN;
    END

    BEGIN TRY
        OPEN SYMMETRIC KEY SK_DriverLicense DECRYPTION BY CERTIFICATE
Cert_DriverLicense;
        INSERT INTO Drivers (FullName, LicenseNumber, Phone,
ExperienceYears)

```

```

VALUES (
    LTRIM(RTRIM(@FullName)),
    EncryptByKey(Key_GUID('SK_DriverLicense'),
@LicenseNumber),
    @Phone,
    @ExperienceYears
);
CLOSE SYMMETRIC KEY SK_DriverLicense;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    IF EXISTS (SELECT * FROM sys.openkeys WHERE key_name =
'SK_DriverLicense')
        CLOSE SYMMETRIC KEY SK_DriverLicense;
    DECLARE @ErrMsg NVARCHAR(4000);
    SET @ErrMsg = ERROR_MESSAGE();
    RAISERROR(@ErrMsg, 16, 1);
END CATCH
END
GO

-- Создание водителя (алиас для совместимости с server.js)
IF OBJECT_ID('sp_CreateDriver', 'P') IS NOT NULL DROP PROCEDURE
sp_CreateDriver;
GO
CREATE PROCEDURE sp_CreateDriver
    @FullName NVARCHAR(100),
    @LicenseNumber NVARCHAR(50),
    @Phone NVARCHAR(20),
    @ExperienceYears INT,
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            SELECT 0 AS Success, NULL AS ID, N'Доступ запрещен.
Требуются права администратора.' AS Message;
            RETURN;
        END
    END

    BEGIN TRY
        EXEC sp_AddDriver @FullName, @LicenseNumber, @Phone,
@ExperienceYears;
    
```

```

        SELECT 1 AS Success, SCOPE_IDENTITY() AS ID, N'Водитель
успешно добавлен.' AS Message;
    END TRY
    BEGIN CATCH
        DECLARE @ErrMsg NVARCHAR(4000);
        SET @ErrMsg = ERROR_MESSAGE();
        SELECT 0 AS Success, NULL AS ID, @ErrMsg AS Message;
    END CATCH
END
GO

-- Получение информации о водителях для обычных пользователей (с
ручным маскированием)
-- Применяем маскирование вручную, так как пользователь БД может
иметь права UNMASK
IF OBJECT_ID('sp_GetDriversForUser', 'P') IS NOT NULL DROP PROCEDURE
sp_GetDriversForUser;
GO
CREATE PROCEDURE sp_GetDriversForUser
AS
BEGIN
    SET NOCOUNT ON;
    -- Обычные пользователи видят:
    -- Телефон полностью (для связи)
    -- Опыт работы полностью
    -- FullName с маскированием: первый символ + "XXXX"

    SELECT
        DriverID,
        -- Применяем маскирование вручную: первый символ + "XXXX"
        CASE
            WHEN LEN(FullName) > 0
            THEN LEFT(FullName, 1) + 'XXXX'
            ELSE 'XXXX'
        END AS FullName,
        Phone,          -- Полностью видно
        ExperienceYears, -- Полностью видно
        IsActive
    FROM Drivers
    WHERE IsActive = 1
    ORDER BY FullName;
END
GO

-- Получение всех водителей для админа (с зашифрованной лицензией в
виде шифра для демонстрации)
IF OBJECT_ID('sp_GetAllDrivers', 'P') IS NOT NULL DROP PROCEDURE
sp_GetAllDrivers;
GO
CREATE PROCEDURE sp_GetAllDrivers
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS

```

```

BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    -- Админ видит все данные без маскирования (FullName не
маскируется для админа)
    -- Включая зашифрованную лицензию в виде шифра (для демонстрации
шифрования)
    -- Полная расшифровка только через
sp_GetDriverLicenseWithPassword с паролем
    SELECT
        DriverID,
        FullName, -- Для админа маскирование не применяется
        Phone,
        ExperienceYears,
        IsActive,
        -- Показываем зашифрованную лицензию в виде hex-строки для
демонстрации шифрования
        CASE
            WHEN LicenseNumber IS NOT NULL
            THEN ' ENCRYPTED: ' + SUBSTRING(CONVERT(NVARCHAR(MAX),
LicenseNumber, 2), 1, 32) + '...'
            ELSE NULL
        END AS LicenseNumber
    FROM Drivers
    ORDER BY FullName;
END
GO

-- Получение лицензии водителя с проверкой пароля админа
IF OBJECT_ID('sp_GetDriverLicenseWithPassword', 'P') IS NOT NULL DROP
PROCEDURE sp_GetDriverLicenseWithPassword;
GO
CREATE PROCEDURE sp_GetDriverLicenseWithPassword
    @DriverID INT,
    @AdminPassword NVARCHAR(128)
AS
BEGIN
    SET NOCOUNT ON;

```

```

-- Проверка существования водителя
IF NOT EXISTS (SELECT 1 FROM Drivers WHERE DriverID = @DriverID)
BEGIN
    SELECT 0 AS Success, NULL AS LicenseNumber, N'Водитель не
найден.' AS Message;
    RETURN;
END

-- Проверка пароля админа (ищем любого админа с таким паролем)
IF NOT EXISTS (
    SELECT 1 FROM Users
    WHERE Role = 'admin'
    AND PasswordHash = HASHBYTES('SHA2_256', @AdminPassword)
)
BEGIN
    SELECT 0 AS Success, NULL AS LicenseNumber, N'Неверный пароль
администратора.' AS Message;
    RETURN;
END

-- Если пароль верный, расшифровываем лицензию
BEGIN TRY
    OPEN SYMMETRIC KEY SK_DriverLicense DECRYPTION BY CERTIFICATE
Cert_DriverLicense;

    SELECT
        1 AS Success,
        CONVERT(NVARCHAR(50), DecryptByKey(LicenseNumber)) AS
LicenseNumber,
        N'Лицензия успешно получена.' AS Message
    FROM Drivers
    WHERE DriverID = @DriverID;

    CLOSE SYMMETRIC KEY SK_DriverLicense;
END TRY
BEGIN CATCH
    IF EXISTS (SELECT * FROM sys.openkeys WHERE key_name =
'SK_DriverLicense')
        CLOSE SYMMETRIC KEY SK_DriverLicense;
    DECLARE @ErrMsg NVARCHAR(4000);
    SET @ErrMsg = ERROR_MESSAGE();
    SELECT 0 AS Success, NULL AS LicenseNumber, @ErrMsg AS
Message;
END CATCH
END
GO

-- Обновление данных водителя (с валидацией)
IF OBJECT_ID('sp_UpdateDriver', 'P') IS NOT NULL DROP PROCEDURE
sp_UpdateDriver;
GO
CREATE PROCEDURE sp_UpdateDriver
    @DriverID INT,

```

```

@FullName NVARCHAR(100),
@LicenseNumber NVARCHAR(50),
@Phone NVARCHAR(20),
@ExperienceYears INT,
@IsActive BIT,
@AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;
    DECLARE @ErrMsg NVARCHAR(4000);
    DECLARE @EncryptedLicense VARBINARY(MAX);

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    IF NOT EXISTS (SELECT 1 FROM Drivers WHERE DriverID = @DriverID)
    BEGIN
        RAISERROR(N'Водитель не найден.', 16, 1);
        RETURN;
    END

    IF @ExperienceYears IS NOT NULL AND @ExperienceYears < 0
    BEGIN
        RAISERROR(N'Опыт работы не может быть отрицательным.', 16,
1);
        RETURN;
    END

    IF @LicenseNumber IS NOT NULL
    BEGIN
        BEGIN TRY
            OPEN SYMMETRIC KEY SK_DriverLicense DECRYPTION BY
CERTIFICATE Cert_DriverLicense;
            SET @EncryptedLicense =
EncryptByKey(Key_GUID('SK_DriverLicense'), @LicenseNumber);
            CLOSE SYMMETRIC KEY SK_DriverLicense;
        END TRY
        BEGIN CATCH
            IF EXISTS (SELECT * FROM sys.openkeys WHERE key_name =
'SK_DriverLicense')
                CLOSE SYMMETRIC KEY SK_DriverLicense;
        END CATCH
    END

```

```

        SET @ErrMsg = ERROR_MESSAGE();
        RAISERROR(@ErrMsg, 16, 1);
        RETURN;
    END CATCH
END

BEGIN TRY
    UPDATE Drivers
    SET
        FullName = ISNULL(LTRIM(RTRIM(@FullName)), FullName),
        LicenseNumber = ISNULL(@EncryptedLicense, LicenseNumber),
        Phone = ISNULL(@Phone, Phone),
        ExperienceYears = ISNULL(@ExperienceYears,
ExperienceYears),
        IsActive = ISNULL(@IsActive, IsActive)
    WHERE DriverID = @DriverID;

    SELECT 1 AS Success, N'Водитель успешно обновлен.' AS
Message;
END TRY
BEGIN CATCH
    SET @ErrMsg = ERROR_MESSAGE();
    SELECT 0 AS Success, @ErrMsg AS Message;
END CATCH
END
GO

-- Удаление водителя
IF OBJECT_ID('sp_DeleteDriver', 'P') IS NOT NULL DROP PROCEDURE
sp_DeleteDriver;
GO
CREATE PROCEDURE sp_DeleteDriver
    @DriverID INT,
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END
END

```

```

    IF NOT EXISTS (SELECT 1 FROM Drivers WHERE DriverID = @DriverID)
    BEGIN
        RAISERROR(N'Водитель не найден.', 16, 1);
        RETURN;
    END

    IF EXISTS (SELECT 1 FROM Trips WHERE DriverID = @DriverID)
    BEGIN
        RAISERROR(N'Невозможно удалить водителя, так как он назначен
на рейсы.', 16, 1);
        RETURN;
    END

    DELETE FROM Drivers WHERE DriverID = @DriverID;
END
GO

-- ХРАНИМЫЕ ПРОЦЕДУРЫ ДЛЯ УПРАВЛЕНИЯ МАРШРУТАМИ

-- Добавление маршрута (с валидацией)
IF OBJECT_ID('sp_AddRoute', 'P') IS NOT NULL DROP PROCEDURE
sp_AddRoute;
GO
CREATE PROCEDURE sp_AddRoute
    @RouteName NVARCHAR(100),
    @DepartureCity NVARCHAR(50),
    @ArrivalCity NVARCHAR(50),
    @DistanceKM INT,
    @DurationMinutes INT,
    @TotalSeats INT,
    @BasePrice DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;

    -- Валидация
    IF @RouteName IS NULL OR LEN(LTRIM(RTRIM(@RouteName))) = 0
    BEGIN
        RAISERROR(N'Название маршрута не может быть пустым.', 16, 1);
        RETURN;
    END

    IF @DepartureCity IS NULL OR LEN(LTRIM(RTRIM(@DepartureCity))) =
0
    BEGIN
        RAISERROR(N'Город отправления не может быть пустым.', 16, 1);
        RETURN;
    END

    IF @ArrivalCity IS NULL OR LEN(LTRIM(RTRIM(@ArrivalCity))) = 0
    BEGIN
        RAISERROR(N'Город прибытия не может быть пустым.', 16, 1);
        RETURN;
    END

```

```

END

IF @DepartureCity = @ArrivalCity
BEGIN
    RAISERROR(N'Город отправления и прибытия не могут
совпадать.', 16, 1);
    RETURN;
END

IF @DistanceKM IS NULL OR @DistanceKM <= 0
BEGIN
    RAISERROR(N'Расстояние должно быть больше 0.', 16, 1);
    RETURN;
END

IF @DurationMinutes IS NULL OR @DurationMinutes <= 0
BEGIN
    RAISERROR(N'Длительность должна быть больше 0.', 16, 1);
    RETURN;
END

IF @TotalSeats IS NULL OR @TotalSeats < 10 OR @TotalSeats > 60
BEGIN
    RAISERROR(N'Количество мест должно быть от 10 до 60.', 16,
1);
    RETURN;
END

IF @BasePrice IS NULL OR @BasePrice <= 0
BEGIN
    RAISERROR(N'Базовая цена должна быть больше 0.', 16, 1);
    RETURN;
END

INSERT INTO Routes (RouteName, DepartureCity, ArrivalCity,
DistanceKM, DurationMinutes, TotalSeats, BasePrice)
VALUES (@RouteName, @DepartureCity, @ArrivalCity, @DistanceKM,
@DurationMinutes, @TotalSeats, @BasePrice);
END
GO

-- Создание маршрута (алиас для совместимости с server.js)
IF OBJECT_ID('sp_CreateRoute', 'P') IS NOT NULL DROP PROCEDURE
sp_CreateRoute;
GO
CREATE PROCEDURE sp_CreateRoute
    @RouteName NVARCHAR(100),
    @DepartureCity NVARCHAR(50),
    @ArrivalCity NVARCHAR(50),
    @DistanceKm INT,
    @DurationMinutes INT,
    @BasePrice DECIMAL(10,2),
    @AdminUserID INT -- ID администратора для проверки прав (NULL

```

```

если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;
    DECLARE @TotalSeats INT;
    SET @TotalSeats = 50; -- Значение по умолчанию

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            SELECT 0 AS Success, NULL AS ID, N'Доступ запрещен.
Требуется права администратора.' AS Message;
            RETURN;
        END
    END

    BEGIN TRY
        EXEC sp_AddRoute @RouteName, @DepartureCity, @ArrivalCity,
@DistanceKm, @DurationMinutes, @TotalSeats, @BasePrice;
        SELECT 1 AS Success, SCOPE_IDENTITY() AS ID, N'Маршрут
успешно добавлен.' AS Message;
    END TRY
    BEGIN CATCH
        DECLARE @ErrMsg NVARCHAR(4000);
        SET @ErrMsg = ERROR_MESSAGE();
        SELECT 0 AS Success, NULL AS ID, @ErrMsg AS Message;
    END CATCH
END
GO

-- Обновление маршрута (с валидацией)
IF OBJECT_ID('sp_UpdateRoute', 'P') IS NOT NULL DROP PROCEDURE
sp_UpdateRoute;
GO
CREATE PROCEDURE sp_UpdateRoute
    @RouteID INT,
    @RouteName NVARCHAR(100),
    @DepartureCity NVARCHAR(50),
    @ArrivalCity NVARCHAR(50),
    @DistanceKM INT,
    @DurationMinutes INT,
    @TotalSeats INT,
    @BasePrice DECIMAL(10,2),
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;

```

```

DECLARE @IsAdmin BIT;
SET @IsAdmin = 0;

-- Проверка прав администратора
IF @AdminUserID IS NOT NULL
BEGIN
    EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
    IF @IsAdmin = 0
    BEGIN
        RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
        RETURN;
    END
END

IF NOT EXISTS (SELECT 1 FROM Routes WHERE RouteID = @RouteID)
BEGIN
    RAISERROR(N'Маршрут не найден.', 16, 1);
    RETURN;
END

IF (@DepartureCity IS NOT NULL AND @ArrivalCity IS NOT NULL) AND
@DepartureCity = @ArrivalCity
BEGIN
    RAISERROR(N'Город отправления и прибытия не могут
совпадать.', 16, 1);
    RETURN;
END

IF @DistanceKM IS NOT NULL AND @DistanceKM <= 0
BEGIN
    RAISERROR(N'Расстояние должно быть больше 0.', 16, 1);
    RETURN;
END

IF @DurationMinutes IS NOT NULL AND @DurationMinutes <= 0
BEGIN
    RAISERROR(N'Длительность должна быть больше 0.', 16, 1);
    RETURN;
END

IF @TotalSeats IS NOT NULL AND (@TotalSeats < 10 OR @TotalSeats >
60)
BEGIN
    RAISERROR(N'Количество мест должно быть от 10 до 60.', 16,
1);
    RETURN;
END

IF @BasePrice IS NOT NULL AND @BasePrice <= 0
BEGIN
    RAISERROR(N'Базовая цена должна быть больше 0.', 16, 1);
    RETURN;

```

```

END

BEGIN TRY
    UPDATE Routes
    SET
        RouteName = ISNULL(@RouteName, RouteName),
        DepartureCity = ISNULL(@DepartureCity, DepartureCity),
        ArrivalCity = ISNULL(@ArrivalCity, ArrivalCity),
        DistanceKM = ISNULL(@DistanceKM, DistanceKM),
        DurationMinutes = ISNULL(@DurationMinutes,
DurationMinutes),
        TotalSeats = ISNULL(@TotalSeats, TotalSeats),
        BasePrice = ISNULL(@BasePrice, BasePrice)
    WHERE RouteID = @RouteID;

    SELECT 1 AS Success, N'Маршрут успешно обновлен.' AS Message;
END TRY
BEGIN CATCH
    DECLARE @ErrMsg NVARCHAR(4000);
    SET @ErrMsg = ERROR_MESSAGE();
    SELECT 0 AS Success, @ErrMsg AS Message;
END CATCH
END
GO

-- Удаление маршрута
IF OBJECT_ID('sp_DeleteRoute', 'P') IS NOT NULL DROP PROCEDURE
sp_DeleteRoute;
GO
CREATE PROCEDURE sp_DeleteRoute
    @RouteID INT,
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END
END

IF NOT EXISTS (SELECT 1 FROM Routes WHERE RouteID = @RouteID)
BEGIN

```

```

        RAISERROR(N'Маршрут не найден.', 16, 1);
        RETURN;
    END

    -- Каскадное удаление сработает автоматически для Trips и
    Bookings
    DELETE FROM Routes WHERE RouteID = @RouteID;
END
GO

-- Получение списка маршрутов
IF OBJECT_ID('sp_GetRoutesList', 'P') IS NOT NULL DROP PROCEDURE
sp_GetRoutesList;
GO
CREATE PROCEDURE sp_GetRoutesList
AS
BEGIN
    SET NOCOUNT ON;
    SELECT RouteID, RouteName, DepartureCity, ArrivalCity,
DistanceKM, DurationMinutes, TotalSeats, BasePrice
    FROM Routes
    ORDER BY DepartureCity, ArrivalCity;
END
GO

-- Получение всех маршрутов (алиас)
IF OBJECT_ID('sp_GetRoutes', 'P') IS NOT NULL DROP PROCEDURE
sp_GetRoutes;
GO
CREATE PROCEDURE sp_GetRoutes
AS
BEGIN
    SET NOCOUNT ON;
    EXEC sp_GetRoutesList;
END
GO

-- Получение всех маршрутов (для админа)
IF OBJECT_ID('sp_GetAllRoutes', 'P') IS NOT NULL DROP PROCEDURE
sp_GetAllRoutes;
GO
CREATE PROCEDURE sp_GetAllRoutes
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN

```

```

EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
IF @IsAdmin = 0
BEGIN
    RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
    RETURN;
END
END

EXEC sp_GetRoutesList;
END
GO

-- Улучшенный поиск маршрутов с датой (для пользователя) - с
информацией о водителе
-- Применяем маскирование вручную для обычных пользователей
IF OBJECT_ID('sp_SearchRoutes', 'P') IS NOT NULL DROP PROCEDURE
sp_SearchRoutes;
GO
CREATE PROCEDURE sp_SearchRoutes
    @DepartureCity NVARCHAR(50) = NULL,
    @ArrivalCity NVARCHAR(50) = NULL,
    @TripDate DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Поиск маршрутов с фильтрацией по городам и дате, включая
информацию о водителе (маскированную)
    SELECT DISTINCT
        R.RouteID,
        R.RouteName,
        R.DepartureCity,
        R.ArrivalCity,
        R.DistanceKM,
        R.DurationMinutes,
        R.TotalSeats,
        R.BasePrice,
        T.TripID,
        T.TripDate,
        T.DepartureTime,
        T.AvailableSeats,
        T.Status,
        T.DriverID,
        -- Применяем маскирование вручную: первый символ + "XXXX"
CASE
    WHEN LEN(D.FullName) > 0
    THEN LEFT(D.FullName, 1) + 'XXXX'
    ELSE 'XXXX'
END AS DriverFullName,
        D.Phone AS DriverPhone,
        D.ExperienceYears AS DriverExperience
    FROM Routes R

```

```

LEFT JOIN Trips T ON R.RouteID = T.RouteID
LEFT JOIN Drivers D ON T.DriverID = D.DriverID
WHERE
    (@DepartureCity IS NULL OR R.DepartureCity LIKE '%' +
@DepartureCity + '%')
    AND (@ArrivalCity IS NULL OR R.ArrivalCity LIKE '%' +
@ArrivalCity + '%')
    AND (@TripDate IS NULL OR T.TripDate = @TripDate)
    AND (T.Status IS NULL OR T.Status != 'cancelled')
ORDER BY R.BasePrice, T.TripDate, T.DepartureTime;
END
GO

-- ХРАНИМЫЕ ПРОЦЕДУРЫ ДЛЯ УПРАВЛЕНИЯ РЕЙСАМИ

-- Добавление рейса (с валидацией)
IF OBJECT_ID('sp_AddTrip', 'P') IS NOT NULL DROP PROCEDURE
sp_AddTrip;
GO
CREATE PROCEDURE sp_AddTrip
    @RouteID INT,
    @DriverID INT,
    @TripDate DATE,
    @DepartureTime TIME
AS
BEGIN
    SET NOCOUNT ON;

    -- Валидация
    IF NOT EXISTS (SELECT 1 FROM Routes WHERE RouteID = @RouteID)
    BEGIN
        RAISERROR(N'Маршрут с указанным RouteID не найден.', 16, 1);
        RETURN;
    END

    IF NOT EXISTS (SELECT 1 FROM Drivers WHERE DriverID = @DriverID
AND IsActive = 1)
    BEGIN
        RAISERROR(N'Водитель не найден или неактивен.', 16, 1);
        RETURN;
    END

    IF @TripDate IS NULL OR @TripDate < CAST(GETDATE() AS DATE)
    BEGIN
        RAISERROR(N'Дата рейса не может быть в прошлом.', 16, 1);
        RETURN;
    END

    DECLARE @TotalSeats INT;
    SELECT @TotalSeats = TotalSeats FROM Routes WHERE RouteID =
@RouteID;

    INSERT INTO Trips (RouteID, DriverID, TripDate, DepartureTime,

```

```

TotalSeats)
    VALUES (@RouteID, @DriverID, @TripDate, @DepartureTime,
@TotalSeats);
END
GO

-- Создание рейса (алиас для совместимости с server.js)
IF OBJECT_ID('sp_CreateTrip', 'P') IS NOT NULL DROP PROCEDURE
sp_CreateTrip;
GO
CREATE PROCEDURE sp_CreateTrip
    @RouteID INT,
    @DriverID INT,
    @BusID INT, -- Не используется, но оставлен для совместимости
    @TripDate DATE,
    @DepartureTime NVARCHAR(8), -- Принимаем как строку для более
гибкой обработки
    @TotalSeats INT, -- Не используется, берется из маршрута
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            SELECT 0 AS Success, NULL AS ID, N'Доступ запрещен.
Требуются права администратора.' AS Message;
            RETURN;
        END
    END

    -- Преобразуем строку времени в TIME тип
    DECLARE @DepartureTimeAsTime TIME;
    BEGIN TRY
        SET @DepartureTimeAsTime = CAST(@DepartureTime AS TIME);
    END TRY
    BEGIN CATCH
        SELECT 0 AS Success, NULL AS ID, N'Неверный формат времени.
Ожидается HH:mm:ss' AS Message;
        RETURN;
    END CATCH

    BEGIN TRY
        EXEC sp_AddTrip @RouteID, @DriverID, @TripDate,
@DepartureTimeAsTime;
        SELECT 1 AS Success, SCOPE_IDENTITY() AS ID, N'Рейс успешно

```

```

добавлен.' AS Message;
END TRY
BEGIN CATCH
    DECLARE @ErrMsg NVARCHAR(4000);
    SET @ErrMsg = ERROR_MESSAGE();
    SELECT 0 AS Success, NULL AS ID, @ErrMsg AS Message;
END CATCH
END
GO

-- Обновление рейса (с валидацией)
IF OBJECT_ID('sp_UpdateTrip', 'P') IS NOT NULL DROP PROCEDURE
sp_UpdateTrip;
GO
CREATE PROCEDURE sp_UpdateTrip
    @TripID INT,
    @RouteID INT,
    @DriverID INT,
    @BusID INT, -- Не используется
    @TripDate DATE,
    @DepartureTime NVARCHAR(8), -- Принимаем как строку для более
гибкой обработки
    @TotalSeats INT, -- Не используется, берется из маршрута
    @Status NVARCHAR(20),
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;
    DECLARE @ErrMsg NVARCHAR(4000);

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    IF NOT EXISTS (SELECT 1 FROM Trips WHERE TripID = @TripID)
    BEGIN
        RAISERROR(N'Рейс не найден.', 16, 1);
        RETURN;
    END

    IF @RouteID IS NOT NULL AND NOT EXISTS (SELECT 1 FROM Routes
WHERE RouteID = @RouteID)

```

```

BEGIN
    RAISERROR(N'Маршрут не найден.', 16, 1);
    RETURN;
END

    IF @DriverID IS NOT NULL AND NOT EXISTS (SELECT 1 FROM Drivers
WHERE DriverID = @DriverID AND IsActive = 1)
    BEGIN
        RAISERROR(N'Водитель не найден или неактивен.', 16, 1);
        RETURN;
    END

    IF @Status IS NOT NULL AND @Status NOT IN ('planned',
'in_progress', 'completed', 'cancelled')
    BEGIN
        RAISERROR(N'Неверный статус рейса.', 16, 1);
        RETURN;
    END

    DECLARE @NewTotalSeats INT;
    IF @RouteID IS NOT NULL
    BEGIN
        SELECT @NewTotalSeats = TotalSeats FROM Routes WHERE RouteID
= @RouteID;
    END

    -- Преобразуем строку времени в TIME тип, если она указана
    DECLARE @DepartureTimeAsTime TIME = NULL;
    IF @DepartureTime IS NOT NULL AND @DepartureTime != ''
    BEGIN
        BEGIN TRY
            SET @DepartureTimeAsTime = CAST(@DepartureTime AS TIME);
        END TRY
        BEGIN CATCH
            RAISERROR(N'Неверный формат времени. Ожидается HH:mm:ss',
16, 1);
            RETURN;
        END CATCH
    END

    BEGIN TRY
        UPDATE Trips
        SET
            RouteID = ISNULL(@RouteID, RouteID),
            DriverID = ISNULL(@DriverID, DriverID),
            TripDate = ISNULL(@TripDate, TripDate),
            DepartureTime = ISNULL(@DepartureTimeAsTime,
DepartureTime),
            Status = ISNULL(@Status, Status),
            TotalSeats = ISNULL(@NewTotalSeats, TotalSeats)
        WHERE TripID = @TripID;

        SELECT 1 AS Success, N'Рейс успешно обновлен.' AS Message;
    
```

```

        END TRY
    BEGIN CATCH
        SET @ErrMsg = ERROR_MESSAGE();
        SELECT 0 AS Success, @ErrMsg AS Message;
    END CATCH
END
GO

-- Удаление рейса
IF OBJECT_ID('sp_DeleteTrip', 'P') IS NOT NULL DROP PROCEDURE
sp_DeleteTrip;
GO
CREATE PROCEDURE sp_DeleteTrip
    @TripID INT,
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    IF NOT EXISTS (SELECT 1 FROM Trips WHERE TripID = @TripID)
    BEGIN
        RAISERROR(N'Рейс не найден.', 16, 1);
        RETURN;
    END

    -- Каскадное удаление сработает автоматически для Bookings
    DELETE FROM Trips WHERE TripID = @TripID;
END
GO

-- Получение рейсов по маршруту
-- Применяем маскирование вручную для обычных пользователей
IF OBJECT_ID('sp_GetTripsByRoute', 'P') IS NOT NULL DROP PROCEDURE
sp_GetTripsByRoute;
GO
CREATE PROCEDURE sp_GetTripsByRoute
    @RouteID INT
AS

```

```

BEGIN
    SET NOCOUNT ON;

    SELECT
        T.TripID,
        T.RouteID,
        R.RouteName,
        T.DriverID,
        -- Применяем маскирование вручную: первый символ + "XXXX"
        CASE
            WHEN LEN(D.FullName) > 0
            THEN LEFT(D.FullName, 1) + 'XXXX'
            ELSE 'XXXX'
        END AS DriverName,
        T.TripDate,
        T.DepartureTime,
        T.TotalSeats,
        T.BookingSeats,
        T.AvailableSeats,
        T.Status,
        R.BasePrice
    FROM Trips T
    JOIN Routes R ON T.RouteID = R.RouteID
    JOIN Drivers D ON T.DriverID = D.DriverID
    WHERE T.RouteID = @RouteID
    AND T.Status != 'cancelled'
    ORDER BY T.TripDate, T.DepartureTime;
END
GO

-- Получение всех рейсов (для админа)
IF OBJECT_ID('sp_GetAllTrips', 'P') IS NOT NULL DROP PROCEDURE
sp_GetAllTrips;
GO
CREATE PROCEDURE sp_GetAllTrips
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

```

```

END

SELECT
    T.TripID,
    T.RouteID,
    R.RouteName,
    T.DriverID,
    D.FullName AS DriverName,
    T.TripDate,
    CAST(T.DepartureTime AS NVARCHAR(8)) AS DepartureTime, --
Преобразуем TIME в строку для удобной обработки
    T.TotalSeats,
    T.BookedSeats,
    T.AvailableSeats,
    T.Status
FROM Trips T
JOIN Routes R ON T.RouteID = R.RouteID
JOIN Drivers D ON T.DriverID = D.DriverID
ORDER BY T.TripDate DESC, T.DepartureTime DESC;
END
GO

-- Проверка доступности мест на рейсе
IF OBJECT_ID('sp_GetTripAvailability', 'P') IS NOT NULL DROP
PROCEDURE sp_GetTripAvailability;
GO
CREATE PROCEDURE sp_GetTripAvailability
    @TripID INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Trips WHERE TripID = @TripID)
    BEGIN
        RETURN;
    END

    SELECT
        BookingID,
        SeatNumber,
        IsCancelled
    FROM Bookings
    WHERE TripID = @TripID
    ORDER BY SeatNumber;
END
GO

-- ХРАНИМЫЕ ПРОЦЕДУРЫ ДЛЯ БРОНИРОВАНИЯ

-- Бронирование билета (с валидацией)
IF OBJECT_ID('sp_BookTicket', 'P') IS NOT NULL DROP PROCEDURE
sp_BookTicket;

```

```

GO
CREATE PROCEDURE sp_BookTicket
    @TripID INT,
    @UserID INT,
    @SeatNumber INT,
    @PassengerPhone NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    -- Валидация
    IF @PassengerPhone IS NULL OR LEN(LTRIM(RTRIM(@PassengerPhone)))
= 0
    BEGIN
        SELECT 0 AS Success, N'Телефон пассажира не может быть
пустым.' AS Message;
        RETURN;
    END

    IF @SeatNumber IS NULL OR @SeatNumber < 1
    BEGIN
        SELECT 0 AS Success, N'Номер места должен быть больше 0.' AS
Message;
        RETURN;
    END

    DECLARE @TotalSeats INT;
    DECLARE @BasePrice DECIMAL(10,2);
    DECLARE @TripStatus NVARCHAR(20);

    SELECT
        @BasePrice = R.BasePrice,
        @TotalSeats = T.TotalSeats,
        @TripStatus = T.Status
    FROM Trips T
    JOIN Routes R ON T.RouteID = R.RouteID
    WHERE T.TripID = @TripID;

    IF @BasePrice IS NULL
    BEGIN
        SELECT 0 AS Success, N'Рейс не найден.' AS Message;
        RETURN;
    END

    IF @TripStatus = 'cancelled'
    BEGIN
        SELECT 0 AS Success, N'Рейс отменен, бронирование
невозможно.' AS Message;
        RETURN;
    END

    IF @SeatNumber > @TotalSeats
    BEGIN

```

```

        SELECT 0 AS Success, N'Указано недействительное место для
этого рейса.' AS Message;
        RETURN;
    END

    IF EXISTS (SELECT 1 FROM Bookings WHERE TripID = @TripID AND
SeatNumber = @SeatNumber AND IsCancelled = 0)
    BEGIN
        SELECT 0 AS Success, N'Место уже занято.' AS Message;
        RETURN;
    END

    BEGIN TRANSACTION
    BEGIN TRY
        INSERT INTO Bookings (TripID, UserID, SeatNumber,
PassengerPhone, PricePaid)
        VALUES (@TripID, @UserID, @SeatNumber, @PassengerPhone,
@BasePrice);

        UPDATE Trips SET BookedSeats = BookedSeats + 1 WHERE TripID =
@TripID;

        COMMIT TRANSACTION;
        SELECT 1 AS Success, N'Билет успешно забронирован.' AS
Message;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        DECLARE @ErrMsg NVARCHAR(4000);
        SET @ErrMsg = ERROR_MESSAGE();
        SELECT 0 AS Success, @ErrMsg AS Message;
    END CATCH
END
GO

-- Просмотр бронирований для конкретного пользователя
IF OBJECT_ID('sp_GetUserBookings', 'P') IS NOT NULL DROP PROCEDURE
sp_GetUserBookings;
GO
CREATE PROCEDURE sp_GetUserBookings
    @UserID INT,
    @ShowCancelled BIT = 0
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        B.BookingID,
        T.TripID,
        R.RouteID,
        R.RouteName,
        R.DepartureCity,
        R.ArrivalCity,

```

```

        T.TripDate,
        T.DepartureTime,
        B.SeatNumber,
        B.PassengerPhone,
        B.PricePaid,
        B.IsCancelled
FROM Bookings B
JOIN Trips T ON B.TripID = T.TripID
JOIN Routes R ON T.RouteID = R.RouteID
WHERE B.UserID = @UserID
AND (@ShowCancelled = 1 OR B.IsCancelled = 0)
ORDER BY T.TripDate, T.DepartureTime;
END
GO

-- Отмена бронирования (транзакционная)
IF OBJECT_ID('sp_CancelBooking', 'P') IS NOT NULL DROP PROCEDURE
sp_CancelBooking;
GO
CREATE PROCEDURE sp_CancelBooking
    @BookingID BIGINT,
    @UserID INT = NULL -- NULL для админа, значение для пользователя
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @TripID INT;
    DECLARE @IsCancelled BIT;
    DECLARE @ActualUserID INT;

    SELECT
        @TripID = TripID,
        @IsCancelled = IsCancelled,
        @ActualUserID = UserID
    FROM Bookings
    WHERE BookingID = @BookingID;

    IF @TripID IS NULL
    BEGIN
        SELECT 0 AS Success, N'Бронирование не найдено.' AS Message;
        RETURN;
    END

    IF @UserID IS NOT NULL AND @ActualUserID != @UserID
    BEGIN
        SELECT 0 AS Success, N'Вы не являетесь владельцем этого
бронирования.' AS Message;
        RETURN;
    END

    IF @IsCancelled = 1
    BEGIN
        SELECT 0 AS Success, N'Это бронирование уже отменено.' AS
Message;

```

```

        RETURN;
    END

    BEGIN TRANSACTION
    BEGIN TRY
        UPDATE Bookings
        SET IsCancelled = 1
        WHERE BookingID = @BookingID;

        UPDATE Trips
        SET BookedSeats = CASE WHEN BookedSeats > 0 THEN BookedSeats
- 1 ELSE 0 END
        WHERE TripID = @TripID;

        COMMIT TRANSACTION;
        SELECT 1 AS Success, N'Бронирование успешно отменено.' AS
Message;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        DECLARE @ErrMsg NVARCHAR(4000);
        SET @ErrMsg = ERROR_MESSAGE();
        SELECT 0 AS Success, @ErrMsg AS Message;
    END CATCH
END
GO

-- Получение всех бронирований (для админа)
IF OBJECT_ID('sp_GetAllBookings', 'P') IS NOT NULL DROP PROCEDURE
sp_GetAllBookings;
GO
CREATE PROCEDURE sp_GetAllBookings
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    SELECT

```

```

        B.BookingID,
        B.TripID,
        T.RouteID,
        R.RouteName,
        T.TripDate,
        T.DepartureTime,
        B.UserID,
        U.Login AS UserLogin,
        U.FullName AS UserFullName,
        B.SeatNumber,
        B.PassengerPhone,
        B.BookingTime,
        B.PricePaid,
        B.IsCancelled
FROM Bookings B
JOIN Trips T ON B.TripID = T.TripID
JOIN Routes R ON T.RouteID = R.RouteID
JOIN Users U ON B.UserID = U.UserID
ORDER BY B.BookingTime DESC;
END
GO

-- АНАЛИТИЧЕСКИЕ ПРОЦЕДУРЫ

-- Аналитика по маршрутам (для админа)
IF OBJECT_ID('sp_GetRouteAnalysis', 'P') IS NOT NULL DROP PROCEDURE
sp_GetRouteAnalysis;
GO
CREATE PROCEDURE sp_GetRouteAnalysis
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    SELECT
        R.RouteID,
        R.RouteName,
        R.DepartureCity,

```

```

        R.ArrivalCity,
        COUNT(DISTINCT T.TripID) AS TotalTrips,
        COUNT(CASE WHEN B.IsCancelled = 0 THEN B.BookingID END) AS
ActiveBookings,
        COUNT(B.BookingID) AS TotalBookings,
        SUM(CASE WHEN B.IsCancelled = 0 THEN B.PricePaid ELSE 0 END)
AS TotalRevenue,
        AVG(CASE WHEN B.IsCancelled = 0 THEN CAST(B.PricePaid AS
FLOAT) ELSE NULL END) AS AvgBookingPrice,
        R.BasePrice,
        R.TotalSeats,
        SUM(CASE WHEN B.IsCancelled = 0 THEN 1 ELSE 0 END) * 100.0 /
NULLIF(COUNT(DISTINCT T.TripID) * R.TotalSeats, 0) AS OccupancyRate
FROM Routes R
LEFT JOIN Trips T ON R.RouteID = T.RouteID
LEFT JOIN Bookings B ON T.TripID = B.TripID
GROUP BY R.RouteID, R.RouteName, R.DepartureCity, R.ArrivalCity,
R.BasePrice, R.TotalSeats
ORDER BY TotalRevenue DESC;
END
GO

-- Получение общей статистики по системе (для админа)
IF OBJECT_ID('sp_GetSystemStatistics', 'P') IS NOT NULL DROP
PROCEDURE sp_GetSystemStatistics;
GO
CREATE PROCEDURE sp_GetSystemStatistics
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    SELECT
        ISNULL((SELECT COUNT(*) FROM Routes), 0) AS TotalRoutes,
        ISNULL((SELECT COUNT(*) FROM Trips WHERE Status !=
'cancelled'), 0) AS ActiveTrips,
        ISNULL((SELECT COUNT(*) FROM Bookings WHERE IsCancelled = 0),
0) AS ActiveBookings,

```

```

        ISNULL((SELECT COUNT(*) FROM Bookings), 0) AS TotalBookings,
        ISNULL((SELECT SUM(PricePaid) FROM Bookings WHERE IsCancelled
= 0), 0) AS TotalRevenue,
        ISNULL((SELECT COUNT(*) FROM Users WHERE Role = 'user'), 0)
AS TotalUsers,
        ISNULL((SELECT COUNT(*) FROM Drivers WHERE IsActive = 1), 0)
AS ActiveDrivers;
END
GO

-- Массовая вставка бронирований (упрощенная и быстрая версия)
IF OBJECT_ID('sp_BulkInsertBookings', 'P') IS NOT NULL
    DROP PROCEDURE sp_BulkInsertBookings;
GO

CREATE PROCEDURE sp_BulkInsertBookings
    @AdminUserID INT,
    @InsertedRows INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @IsAdmin BIT;
    DECLARE @ErrMsg NVARCHAR(4000);

    SET @InsertedRows = 0;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    IF OBJECT_ID('tempdb..#BulkBookings') IS NULL
    BEGIN
        RAISERROR(N'Временная таблица #BulkBookings не найдена.', 16,
1);
        RETURN;
    END

    BEGIN TRY
        -- Простая массовая вставка
        INSERT INTO Bookings (TripID, UserID, SeatNumber,
PassengerPhone, PricePaid, IsCancelled)
        SELECT
            BB.TripID,

```

```

        BB.UserID,
        BB.SeatNumber,
        BB.PassengerPhone,
        BB.PricePaid,
        0
    FROM #BulkBookings BB
    WHERE NOT EXISTS (
        SELECT 1
        FROM Bookings B
        WHERE B.TripID = BB.TripID
        AND B.SeatNumber = BB.SeatNumber
        AND B.IsCancelled = 0
    );

    SET @InsertedRows = @@ROWCOUNT;
END TRY
BEGIN CATCH
    -- Игнорируем ошибки дубликатов
    IF ERROR_NUMBER() = 2601 OR ERROR_NUMBER() = 2627
    BEGIN
        SET @InsertedRows = 0;
    END
    ELSE
    BEGIN
        SET @ErrMsg = ERROR_MESSAGE();
        RAISERROR(@ErrMsg, 16, 1);
    END
END CATCH
END
GO

```

Приложение Г

```

IF OBJECT_ID('sp_ExportBookingsToJson', 'P') IS NOT NULL DROP
PROCEDURE sp_ExportBookingsToJson;
GO
CREATE PROCEDURE sp_ExportBookingsToJson
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END
    SELECT
        B.BookingID,
        B.TripID,
        T.RouteID,
        R.RouteName,
        B.UserID,
        U.Login AS UserLogin,
        B.SeatNumber,
        B.PassengerPhone,
        B.BookingTime,
        B.PricePaid,
        B.IsCancelled
    FROM Bookings B
    JOIN Trips T ON B.TripID = T.TripID
    JOIN Routes R ON T.RouteID = R.RouteID
    JOIN Users U ON B.UserID = U.UserID
    ORDER BY B.BookingTime DESC;
END
GO

-- Импорт бронирований из JSON (оптимизированная версия)
IF OBJECT_ID('sp_ImportBookingsFromJson', 'P') IS NOT NULL DROP
PROCEDURE sp_ImportBookingsFromJson;
GO
CREATE PROCEDURE sp_ImportBookingsFromJson
    @JsonData NVARCHAR(MAX),
    @AdminUserID INT -- ID администратора для проверки прав (NULL
если не требуется проверка)

```

```

AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IsAdmin BIT;
    SET @IsAdmin = 0;

    -- Проверка прав администратора
    IF @AdminUserID IS NOT NULL
    BEGIN
        EXEC sp_CheckAdminRights @AdminUserID, @IsAdmin OUTPUT;
        IF @IsAdmin = 0
        BEGIN
            RAISERROR(N'Доступ запрещен. Требуются права
администратора.', 16, 1);
            RETURN;
        END
    END

    IF @JsonData IS NULL OR LEN(LTRIM(RTRIM(@JsonData))) = 0
    BEGIN
        RAISERROR(N'JSON данные не предоставлены.', 16, 1);
        RETURN;
    END

    BEGIN TRY
        -- Создаем временную таблицу для batch вставки
        IF OBJECT_ID('tempdb..#BulkBookings') IS NOT NULL DROP TABLE
#BulkBookings;
        CREATE TABLE #BulkBookings (
            TripID INT NOT NULL,
            UserID INT NOT NULL,
            SeatNumber INT NOT NULL,
            PassengerPhone NVARCHAR(20) NOT NULL,
            PricePaid DECIMAL(10,2) NOT NULL,
            INDEX IX_Bulk_Trip_Seat (TripID, SeatNumber)
        );

        -- Загружаем данные из JSON во временную таблицу с ценами
        INSERT INTO #BulkBookings (TripID, UserID, SeatNumber,
PassengerPhone, PricePaid)
        SELECT
            B.TripID,
            B.UserID,
            B.SeatNumber,
            B.PassengerPhone,
            ISNULL(R.BasePrice, 0) AS PricePaid
        FROM OPENJSON(@JsonData, '$.Bookings')
        WITH (
            TripID INT '$.TripID',
            UserID INT '$.UserID',
            SeatNumber INT '$.SeatNumber',
            PassengerPhone NVARCHAR(20) '$.PassengerPhone'
        ) B
    
```

```

INNER JOIN Trips T ON T.TripID = B.TripID
INNER JOIN Routes R ON R.RouteID = T.RouteID
WHERE B.TripID IS NOT NULL
AND B.UserID IS NOT NULL
AND B.SeatNumber > 0
AND B.PassengerPhone IS NOT NULL
AND EXISTS (SELECT 1 FROM Users WHERE UserID = B.UserID);

-- Удаляем дубликаты из временной таблицы
WITH Duplicates AS (
    SELECT TripID, SeatNumber,
           ROW_NUMBER() OVER (PARTITION BY TripID, SeatNumber
ORDER BY (SELECT NULL)) AS rn
    FROM #BulkBookings
)
DELETE FROM #BulkBookings
WHERE EXISTS (
    SELECT 1 FROM Duplicates D
    WHERE D.TripID = #BulkBookings.TripID
    AND D.SeatNumber = #BulkBookings.SeatNumber
    AND D.rn > 1
);

-- Используем процедуру для batch вставки
DECLARE @InsertedRows INT;
EXEC sp_BulkInsertBookings @AdminUserID = @AdminUserID,
@InsertedRows = @InsertedRows OUTPUT;

-- Обрабатываем отмененные бронирования из JSON (если есть)
DECLARE @TripID INT;
DECLARE @UserID INT;
DECLARE @SeatNumber INT;
DECLARE @IsCancelled BIT;

DECLARE cancelled_cursor CURSOR FOR
SELECT
    TripID,
    UserID,
    SeatNumber
FROM OPENJSON(@JsonData, '$.Bookings')
WITH (
    TripID INT '$.TripID',
    UserID INT '$.UserID',
    SeatNumber INT '$.SeatNumber',
    IsCancelled BIT '$.IsCancelled'
)
WHERE TripID IS NOT NULL
AND UserID IS NOT NULL
AND SeatNumber > 0
AND ISNULL(IsCancelled, 0) = 1;

OPEN cancelled_cursor;
FETCH NEXT FROM cancelled_cursor INTO @TripID, @UserID,

```

```

@SeatNumber;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        BEGIN TRY
            DECLARE @BookingID BIGINT;
            SELECT @BookingID = BookingID
            FROM Bookings
            WHERE TripID = @TripID
            AND UserID = @UserID
            AND SeatNumber = @SeatNumber
            AND IsCancelled = 0;

            IF @BookingID IS NOT NULL
            BEGIN
                EXEC sp_CancelBooking @BookingID, NULL;
            END
        END TRY
        BEGIN CATCH
            -- Игнорируем ошибки
        END CATCH

        FETCH NEXT FROM cancelled_cursor INTO @TripID, @UserID,
@SeatNumber;
    END

    CLOSE cancelled_cursor;
    DEALLOCATE cancelled_cursor;

    -- Финальное обновление счетчиков
    UPDATE T
    SET BookedSeats = (
        SELECT COUNT(*)
        FROM Bookings B
        WHERE B.TripID = T.TripID
        AND B.IsCancelled = 0
    )
    FROM Trips T;

    DROP TABLE #BulkBookings;
END TRY
BEGIN CATCH
    IF OBJECT_ID('tempdb..#BulkBookings') IS NOT NULL DROP TABLE
#BulkBookings;
    DECLARE @ErrMsg NVARCHAR(4000);
    SET @ErrMsg = ERROR_MESSAGE();
    RAISERROR(@ErrMsg, 16, 1);
END CATCH
END
GO

```

Приложение Д

```
-- Очистка кэша для чистоты теста
DBCC DROPCLEANBUFFERS;
DBCC FREEPROCCACHE;
GO

-- Включаем статистику
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

DECLARE @StartTime DATETIME2;
DECLARE @EndTime DATETIME2;
DECLARE @Duration BIGINT;
DECLARE @RowCount INT;

-- Создаем таблицу для результатов
IF OBJECT_ID('tempdb..#TestResults') IS NOT NULL DROP TABLE
#TestResults;
CREATE TABLE #TestResults (
    TestNumber INT,
    TestName NVARCHAR(200),
    OperationType NVARCHAR(50),
    DurationMs BIGINT,
    LogicalReads BIGINT,
    PhysicalReads BIGINT,
    RowsReturned INT,
    Notes NVARCHAR(500)
);

PRINT N'ТЕСТ 1: CLUSTERED INDEX SCAN vs INDEX SEEK';
PRINT N'Сравнение поиска по PRIMARY KEY (Clustered) vs Non-Clustered

-- ТЕСТ 1.1: Clustered Index Scan - поиск по диапазону PRIMARY KEY
-- Использует Clustered Index Scan, так как ищем диапазон значений
PRINT N'ТЕСТ 1.1: Clustered Index Scan - поиск по диапазону
BookingID';
PRINT N'Ожидаемая операция: Clustered Index Scan';

-- Очищаем кэш для чистоты теста
DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Запрос, который вызовет Clustered Index Scan
-- Ищем диапазон по PRIMARY KEY (BookingID) - это clustered index
SELECT
    BookingID,
    TripID,
    UserID,
    SeatNumber,
    PricePaid,
    BookingTime
```

```

FROM Bookings
WHERE BookingID BETWEEN 1 AND 50000 -- Диапазон по clustered index
ORDER BY BookingID;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (1, N'Clustered Index Scan - диапазон
BookingID', N'Clustered Index Scan', @Duration, 0, 0, @RowCount,
N'Поиск по диапазону PRIMARY KEY');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

-- ТЕСТ 1.2: Index Seek - поиск по Non-Clustered Index
-- Использует Index Seek, так как есть подходящий non-clustered
индекс
PRINT N'ТЕСТ 1.2: Index Seek - поиск по UserID (Non-Clustered
Index)';
PRINT N'Ожидаемая операция: Index Seek на
IX_Bookings_User_Cancelled';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Запрос, который использует non-clustered index
DECLARE @TestUserID INT = (SELECT TOP 1 UserID FROM Users WHERE Role
= 'user');

SELECT
    BookingID,
    TripID,
    UserID,
    SeatNumber,
    PricePaid,
    BookingTime
FROM Bookings
WHERE UserID = @TestUserID -- Использует IX_Bookings_User_Cancelled
AND IsCancelled = 0
ORDER BY BookingTime DESC;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (2, N'Index Seek - поиск по UserID',
N'Index Seek', @Duration, 0, 0, @RowCount, N'Использует
IX_Bookings_User_Cancelled');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'

```

```

мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

-- ТЕСТ 2: Clustered Index Scan - полное сканирование таблицы
PRINT N'ТЕСТ 2: Clustered Index Scan - полное сканирование';

PRINT N'ТЕСТ 2.1: Clustered Index Scan - COUNT(*) всей таблицы';
PRINT N'Ожидаемая операция: Clustered Index Scan';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Полное сканирование clustered index
SELECT COUNT(*) AS TotalBookings
FROM Bookings;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (3, N'Clustered Index Scan -
COUNT(*)', N'Clustered Index Scan', @Duration, 0, 0, @RowCount,
N'Полное сканирование таблицы');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

PRINT N'ТЕСТ 2.2: Clustered Index Scan - агрегация без WHERE';
PRINT N'Ожидаемая операция: Clustered Index Scan';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Агрегация требует полного сканирования
SELECT
    COUNT(*) AS TotalBookings,
    SUM(PricePaid) AS TotalRevenue,
    AVG(PricePaid) AS AvgPrice,
    MIN(BookingTime) AS FirstBooking,
    MAX(BookingTime) AS LastBooking
FROM Bookings;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (4, N'Clustered Index Scan -
агрегация', N'Clustered Index Scan', @Duration, 0, 0, @RowCount,
N'Агрегация без фильтров');

```

```

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));

-- ТЕСТ 3: Clustered Index Scan vs Index Seek на таблице Trips
PRINT N'ТЕСТ 3: Сравнение на таблице Trips';

PRINT N'ТЕСТ 3.1: Clustered Index Scan - поиск по диапазону TripID';
PRINT N'Ожидаемая операция: Clustered Index Scan';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Диапазон по PRIMARY KEY (clustered index)
SELECT
    TripID,
    RouteID,
    DriverID,
    TripDate,
    DepartureTime,
    Status
FROM Trips
WHERE TripID BETWEEN 1 AND 1000
ORDER BY TripID;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (5, N'Clustered Index Scan - диапазон
TripID', N'Clustered Index Scan', @Duration, 0, 0, @RowCount, N'Поиск
по диапазону PRIMARY KEY');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));

PRINT N'ТЕСТ 3.2: Index Seek - поиск по RouteID и TripDate';
PRINT N'Ожидаемая операция: Index Seek на IX_Trips_Route_Date';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Использует non-clustered index
DECLARE @TestRouteID INT = (SELECT TOP 1 RouteID FROM Routes);
DECLARE @TestDate DATE = CAST(GETDATE() AS DATE);

SELECT
    TripID,
    RouteID,
    TripDate,
    DepartureTime,
    AvailableSeats,

```

```

        Status
FROM Trips
WHERE RouteID = @TestRouteID
AND TripDate >= @TestDate
AND Status = 'planned'
ORDER BY TripDate, DepartureTime;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (6, N'Index Seek - RouteID и
TripDate', N'Index Seek', @Duration, 0, 0, @RowCount, N'Использует
IX_Trips_Route_Date');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

-- TECT 4: Clustered Index Scan - ORDER BY без WHERE
PRINT N'TECT 4: Clustered Index Scan - сортировка';

PRINT N'TECT 4.1: Clustered Index Scan - ORDER BY по PRIMARY KEY';
PRINT N'Ожидаемая операция: Clustered Index Scan (упорядоченное
чтение)';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- ORDER BY по clustered index - может использовать упорядоченное
сканирование
SELECT TOP 1000
    BookingID,
    TripID,
    UserID,
    BookingTime
FROM Bookings
ORDER BY BookingID DESC;  -- Сортировка по PRIMARY KEY

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (7, N'Clustered Index Scan - ORDER BY
BookingID', N'Clustered Index Scan', @Duration, 0, 0, @RowCount,
N'Сортировка по PRIMARY KEY');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));

PRINT N'TECT 4.2: Clustered Index Scan - ORDER BY по

```

```

неиндексированному полю';
PRINT N'Ожидаемая операция: Clustered Index Scan + Sort';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- ORDER BY по полю без индекса - требует сортировки
SELECT TOP 1000
    BookingID,
    TripID,
    UserID,
    BookingTime,
    PricePaid
FROM Bookings
ORDER BY PricePaid DESC;  -- Сортировка по полю без индекса

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (8, N'Clustered Index Scan - ORDER BY
PricePaid', N'Clustered Index Scan + Sort', @Duration, 0, 0,
@RowCount, N'Сортировка по неиндексированному полю');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

-- ТЕСТ 5: Сравнение производительности при JOIN
PRINT N'ТЕСТ 5: Clustered Index Scan в JOIN операциях';

PRINT N'ТЕСТ 5.1: JOIN с Clustered Index Scan';
PRINT N'Ожидаемая операция: Clustered Index Scan на обеих таблицах';
PRINT N'-----';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- JOIN без использования индексов для фильтрации
SELECT TOP 5000
    B.BookingID,
    B.TripID,
    T.TripID AS TripID2,
    T.TripDate,
    B.PricePaid
FROM Bookings B
INNER JOIN Trips T ON B.TripID = T.TripID
WHERE B.BookingID BETWEEN 1 AND 10000  -- Диапазон по clustered index
ORDER BY B.BookingID;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);

```

```

SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (9, N'JOIN с Clustered Index Scan',
N'Clustered Index Scan', @Duration, 0, 0, @RowCount, N'JOIN с
диапазоном по PRIMARY KEY');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

PRINT N'ТЕСТ 5.2: JOIN с Index Seek';
PRINT N'Ожидаемая операция: Index Seek на обеих таблицах';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- JOIN с использованием non-clustered индексов
SELECT
    B.BookingID,
    B.TripID,
    T.TripDate,
    T.DepartureTime,
    B.PricePaid
FROM Bookings B
INNER JOIN Trips T ON B.TripID = T.TripID
WHERE B.UserID = @TestUserID -- Использует
IX_Bookings_User_Cancelled
AND B.IsCancelled = 0
AND T.RouteID = @TestRouteID -- Использует IX_Trips_Route_Date
AND T.Status = 'planned';

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (10, N'JOIN с Index Seek', N'Index
Seek', @Duration, 0, 0, @RowCount, N'JOIN с использованием non-
clustered индексов');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

-- ТЕСТ 6: Производительность при больших объемах данных
PRINT N'ТЕСТ 6: Производительность на больших объемах';

PRINT N'ТЕСТ 6.1: Clustered Index Scan - большой диапазон';
PRINT N'Ожидаемая операция: Clustered Index Scan';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

```

```

-- Большой диапазон - требует полного или почти полного сканирования
DECLARE @MaxBookingID BIGINT = (SELECT MAX(BookingID) FROM Bookings);
DECLARE @RangeStart BIGINT = @MaxBookingID - 50000;

SELECT
    BookingID,
    TripID,
    UserID,
    PricePaid
FROM Bookings
WHERE BookingID >= @RangeStart -- Большой диапазон
ORDER BY BookingID;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (11, N'Clustered Index Scan - большой
диапазон', N'Clustered Index Scan', @Duration, 0, 0, @RowCount,
N'Большой диапазон по PRIMARY KEY');

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

PRINT N'ТЕСТ 6.2: Index Seek - точный поиск';
PRINT N'Ожидаемая операция: Index Seek';

DBCC DROPCLEANBUFFERS;
SET @StartTime = SYSDATETIME();

-- Точный поиск по индексу - намного быстрее
SELECT
    BookingID,
    TripID,
    UserID,
    PricePaid,
    BookingTime
FROM Bookings
WHERE TripID = (SELECT TOP 1 TripID FROM Trips) -- Использует
IX_Bookings_Trip_Time
ORDER BY BookingTime DESC;

SET @EndTime = SYSDATETIME();
SET @Duration = DATEDIFF(MILLISECOND, @StartTime, @EndTime);
SET @RowCount = @@ROWCOUNT;

INSERT INTO #TestResults VALUES (12, N'Index Seek - точный поиск по
TripID', N'Index Seek', @Duration, 0, 0, @RowCount, N'Точный поиск по
IX_Bookings_Trip_Time');

```

```

PRINT N'Время выполнения: ' + CAST(@Duration AS NVARCHAR(20)) + N'
мс';
PRINT N'Найдено записей: ' + CAST(@RowCount AS NVARCHAR(20));
PRINT N'';

-- ИТОГОВАЯ СВОДКА РЕЗУЛЬТАТОВ
PRINT N'ИТОГОВАЯ СВОДКА РЕЗУЛЬТАТОВ';

SELECT
    TestNumber,
    TestName,
    OperationType,
    DurationMs,
    RowsReturned,
    Notes,
    CASE
        WHEN OperationType = 'Clustered Index Scan' THEN N' Медленнее
(сканирование) '
        WHEN OperationType = 'Index Seek' THEN N' Быстрее (точечный
поиск) '
        WHEN OperationType = 'Clustered Index Scan + Sort' THEN N'
Медленнее (сканирование + сортировка) '
        ELSE N' Средне'
    END AS PerformanceIndicator
FROM #TestResults
ORDER BY TestNumber;

PRINT N'АНАЛИЗ РЕЗУЛЬТАТОВ';

-- Статистика по типам операций
SELECT
    OperationType,
    COUNT(*) AS TestCount,
    AVG(CAST(DurationMs AS FLOAT)) AS AvgDurationMs,
    MIN(DurationMs) AS MinDurationMs,
    MAX(DurationMs) AS MaxDurationMs,
    CAST(AVG(CAST(DurationMs AS FLOAT)) AS DECIMAL(10,2)) AS
AvgDurationFormatted
FROM #TestResults
GROUP BY OperationType
ORDER BY AvgDurationMs;

-- Очистка
DROP TABLE #TestResults;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
PRINT N'ТЕСТИРОВАНИЕ ЗАВЕРШЕНО';
GO

```